
LECTURE SLIDES

Python for Data Science

Résoudre, modéliser, vérifier

Bechir Trabelsi — AXA France

Septembre 2026

Architecture du cours

Fondements

1. Syntaxe et méthode
2. Collections et fonctions
3. Algorithmes et complexité
4. Conception objet

Science des données

5. NumPy et calcul numérique
6. pandas et EDA
7. SciPy et simulation
8. Inférence, A/B et PyMC
9. Machine learning

Bonus Hugging Face et IA générative

CHAPITRE 1

Fondamentaux de Python et méthode de résolution

Écrire ses premiers programmes et expliquer leur état

Une question avant le code

Une distance vaut 12 km et une durée 30 minutes.

Question de compréhension

Quelles informations faut-il représenter pour calculer une vitesse en km/h ?

Objectifs

- Prédire la valeur d'une expression courte
- Suivre l'évolution de variables
- Écrire une condition et une boucle simples
- Définir une fonction qui retourne un résultat
- Tester un cas normal et une frontière

Pourquoi Python ?

Un même langage couvre l'algorithme initial, l'analyse de données, le modèle statistique et l'application.

- syntaxe lisible pour suivre l'état du calcul
- bibliothèque standard pour structurer un programme
- bibliothèques scientifiques pour les calculs spécialisés

Langage et noyaux scientifiques

Python orchestre le travail. NumPy, pandas, SciPy, PyMC et scikit-learn fournissent des représentations et algorithmes spécialisés.

Conséquence

Une interface claire en Python peut appeler du code compilé pour les calculs intensifs.

Exploration et programme reproductible

Interpréteur tester une expression courte

Notebook explorer et expliquer une analyse

Script fixer l'ordre d'exécution

Module réutiliser et tester des fonctions

Un notebook doit se relancer depuis un noyau neuf.

Python dans une boîte à outils

- SQL traite les données près de la base
- Python prépare, modélise et automatise
- R offre aussi un environnement statistique riche
- un langage compilé répond à certains besoins de performance

Le choix dépend du problème et du système.

Limites connues dès le départ

- erreurs de type souvent détectées à l'exécution
- environnements et versions à contrôler
- coût mémoire des objets Python
- boucles pures parfois lentes sur de grands tableaux

Tests, profilage et bibliothèques numériques répondent à ces limites.

Installation : le parcours à suivre

1. Installer Python : suivre uniquement la branche Windows ou macOS.
2. Installer VS Code, puis les extensions Python et Jupyter.
3. Créer le dossier du cours et son environnement isolé.
4. Installer Jupyter Notebook et enregistrer le noyau du cours.
5. Exécuter un script et un notebook, puis vérifier leur interpréteur.

Point de départ : un ordinateur avec son système, un navigateur et Internet. Aucune connaissance du terminal n'est supposée.

Avant de télécharger

- Brancher l'ordinateur ; vérifier la connexion Internet et l'espace disque.
- Prévoir le mot de passe du compte local si l'installation le demande.
- Windows : Paramètres → Système → Informations système, puis lire le *type du système* : x64 ou ARM64.
- Mac : menu Apple → À propos de ce Mac ; lire la puce Apple ou le processeur Intel.

Ordinateur géré par l'école ou l'entreprise : demander l'aide du service informatique si une règle bloque l'installation. Ne pas désactiver les protections.

Quel outil fait quoi ?

Outil	Rôle dans notre travail
Python	Exécute les instructions du programme.
VS Code	Éditeur : écrire des fichiers, lancer du code, lire les erreurs.
Terminal	Zone où l'on saisit des commandes du système.
.venv	Dossier contenant un Python isolé et les bibliothèques du projet.
pip	Installe les bibliothèques dans un environnement Python.
Jupyter Notebook	Application qui ouvre les fichiers .ipynb.
Noyau (kernel)	Processus Python qui exécute les cellules d'un notebook.

Installer l'éditeur ne suffit pas : il faut aussi un interpréteur Python.

Python : choisir le téléchargement du cours

1. Ouvrir le navigateur et saisir `python.org/downloads`.
2. Pour ce guide, choisir la série stable **Python 3.13**, puis ouvrir sa page de version.
3. Descendre jusqu'au tableau **Files** et choisir l'installateur de son système.

Lien direct utilisé pour cette édition : Python 3.13.15, tableau Files. Ne pas choisir les archives de sources, *embeddable*, les préversions ou l'option expérimentale *free-threaded*.

Source : Python Software Foundation, téléchargements 3.13.15. Les captures officielles peuvent montrer un autre numéro de version.

Windows 1/3 : ouvrir l'installateur

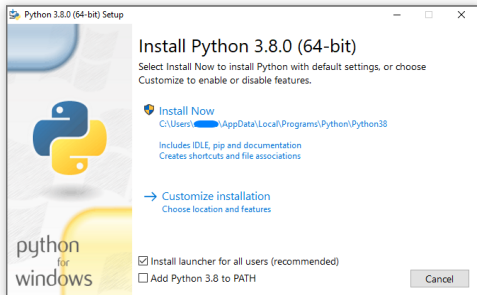
1. Dans le tableau **Files**, cliquer sur **Windows installer (64-bit)** pour un PC x64. Pour ARM64, faire confirmer le choix par l'enseignant.
2. Ouvrir le dossier **Téléchargements** dans l'Explorateur de fichiers.
3. Double-cliquer sur le fichier Python .exe téléchargé depuis python.org.

Le bon parcours

Ce guide utilise l'installateur complet .exe de Python 3.13, pas le bouton *Python install manager* situé au-dessus du tableau.

Source : Python 3.13, Using Python on Windows.

Windows 2/3 : installer Python



1. En bas : cocher **Add Python to PATH**.
2. Garder les options standard, dont pip et le lanceur py.
3. Cliquer sur **Install Now**.
4. Attendre **Setup was successful**, puis **Close**.

Source : Capture officielle : Python Software Foundation. La version affichée est illustrative ; installer 3.13.

Windows 3/3 : vérifier Python et pip

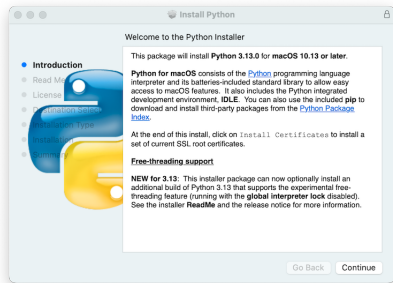
Ouvrir **Démarrer**, saisir **cmd**, puis ouvrir **Invite de commandes**. Saisir une ligne, appuyer sur Entrée, attendre le résultat.

```
py -3.13 --version  
py -3.13 -m pip --version
```

- Première ligne : Python 3.13.x, où x est le correctif installé.
- Deuxième ligne : version de pip, chemin et mention python 3.13.

Fermer et rouvrir le terminal s'il était ouvert avant l'installation. Si py est introuvable, essayer `python -version` et demander de l'aide avant de continuer. Ne pas saisir ces commandes dans une invite `>>>`.

macOS 1/4 : ouvrir le paquet Python

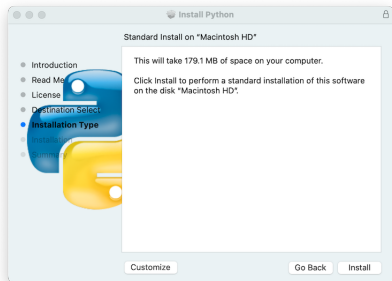


1. Dans **Files**, choisir **macOS installer** : paquet `.pkg` universal2.
2. Finder → Téléchargements : ouvrir le paquet.
3. Cliquer sur **Continuer**, lire les informations et la licence.

Le paquet couvre les Mac Intel et Apple Silicon.

Source : Capture officielle : Python Software Foundation, Using Python on a Mac. Numéro de version illustratif.

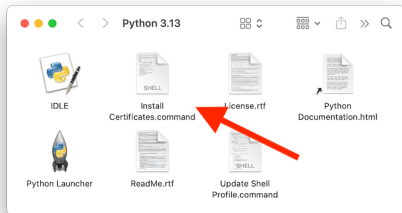
macOS 2/4 : terminer l'installation



1. Si la licence vous convient, l'accepter pour poursuivre.
2. Garder l'installation standard ; cliquer sur **Installer**.
3. Saisir le mot de passe local si demandé.
4. Attendre la confirmation de réussite, puis fermer l'installateur.

Source : Capture officielle : Python Software Foundation. Ne pas modifier le Python fourni par macOS.

macOS 3/4 : installer les certificats



1. Finder → Applications → **Python 3.13**.
2. Double-cliquer sur **Install Certificates.command**.
3. Laisser la fenêtre Terminal travailler jusqu'à la fin.

Cette étape prépare les certificats utilisés pour les connexions HTTPS de cette installation Python.

Source : Capture officielle : Python Software Foundation. Ouvrir le dossier de la version installée.

macOS 4/4 : vérifier Python et pip

Appuyer sur **Cmd + Espace**, saisir **Terminal**, puis Entrée. Saisir chaque commande sur une ligne distincte.

```
python3.13 --version  
python3.13 -m pip --version
```

- Attendu : Python 3.13.x, puis une version de pip pour Python 3.13.
- Si la commande est introuvable : ouvrir un nouveau Terminal et vérifier que l'installation a réussi.

Inutile d'installer Homebrew, Xcode ou Anaconda pour ce parcours. Le mot de passe du Mac ne doit jamais être copié dans le code.

VS Code : télécharger l'éditeur



Ouvrir la page officielle
Télécharger VS Code.

- Windows : **User Installer**, x64 ou Arm64.
- Mac : **Apple silicon**, **Intel chip** ou **Universal**.

Choisir VS Code stable, pas
Visual Studio ni Insiders.

Source : Capture du site officiel Visual Studio Code, 8 septembre 2026. Les prérequis du système figurent sous les boutons.

VS Code : installation selon le système

Windows

1. Ouvrir le .exe depuis Téléchargements.
2. Lire la licence et l'accepter si elle vous convient.
3. Garder le dossier proposé et l'option d'ajout au PATH.
4. **Install**, puis **Finish**. Ouvrir VS Code depuis Démarrer.

macOS

1. Ouvrir le fichier .dmg téléchargé.
2. Glisser **Visual Studio Code.app** vers **Applications**.
3. Ouvrir l'application depuis Applications.
4. Si un .zip a été fourni : le décompresser, puis déplacer l'application au même endroit.

Source : VS Code, Windows setup ; macOS setup.

VS Code : installer les deux extensions



1. Ouvrir **Extensions** : Ctrl + Maj + X (Windows), Cmd + Maj + X (Mac).
2. Chercher ms-python.python : **Python**, éditeur **Microsoft**. Cliquer sur **Install**.
3. Chercher ms-toolsai.jupyter : **Jupyter**, éditeur **Microsoft**. Cliquer sur **Install**.

Un compte Microsoft, Copilot et une extension d'IA ne sont pas nécessaires pour les exercices.

Source : Capture et recherche d'extensions : VS Code, Extension Marketplace ; Jupyter notebooks.

Lire une commande de terminal

Une ligne une commande à saisir, suivie de la touche Entrée.

`cd` change le dossier courant ; `mkdir` crée un dossier.

`python -m pip` demande au Python sélectionné d'exécuter `pip`.

Invite texte affiché avant le curseur : ne pas le recopier.

Deux endroits différents

Les commandes d'installation vont dans le **terminal**, pas dans un fichier Python ni dans une cellule de code. Si `>>>` apparaît, saisir `exit()` pour quitter Python.

Windows : créer le dossier et l'environnement

Dans **Invite de commandes (cmd)**, saisir ces lignes une par une :

```
cd /d "%USERPROFILE%"  
mkdir python-data-science  
cd python-data-science  
py -3.13 -m venv .venv  
.venv\Scripts\activate.bat
```

Le dossier du cours est dans votre dossier utilisateur. `.venv` contient son environnement. L'invite affiche normalement `(.venv)` après activation. Si le dossier existe déjà, ne pas le recréer : entrer dedans avec `cd`.

Source : Python, venv. Ces commandes sont pour cmd, pas PowerShell.

macOS : créer le dossier et l'environnement

Dans **Terminal**, saisir ces lignes une par une :

```
cd ~  
mkdir python-data-science  
cd python-data-science  
python3.13 -m venv .venv  
source .venv/bin/activate
```

~ désigne votre dossier utilisateur. Le point de `.venv` fait partie du nom. L'invite affiche normalement `(.venv)` après activation. Si le dossier existe déjà, entrer dedans sans le recréer.

Source : Python, venv. Commandes pour le terminal macOS (zsh ou bash).

Vérifier l'environnement avant d'installer

```
python --version  
python -c "import sys; print(sys.executable)"  
python -m pip --version
```

- Windows : le chemin Python se termine par `.venv\Scripts \python .exe`.
- Mac : il se termine par `.venv/bin/python`.
- Le chemin de pip doit lui aussi contenir `.venv`.

Le préfixe (`.venv`) est un indice ; le chemin de l'interpréteur est la vérification. Chaque nouveau terminal doit retrouver le dossier et réactiver l'environnement.

Installer Jupyter et les premières bibliothèques

Dans le même terminal, avec `.venv` activé :

```
python -m pip install --upgrade pip
python -m pip install notebook ipykernel numpy matplotlib
python -m pip check
```

- `notebook` : application Jupyter dans le navigateur ; `ipykernel` : noyau Python.
- `numpy`, `matplotlib` : premier test de calcul et de graphique.
- Attendu pour le dernier contrôle : `No broken requirements found`.

Attendre la fin de chaque commande. Les autres bibliothèques seront ajoutées au fil du cours.

Source : Project Jupyter, Installation ; IPython, Installing the kernel.

Enregistrer le noyau du cours

Dans le terminal activé, saisir la commande suivante **sur une seule ligne** :

```
python -m ipykernel install --user --name pyds --display-name "Python (Data Science)"
```

- pyds : identifiant technique de ce noyau.
- Python (Data Science) : nom visible dans le menu de Jupyter.
- Le noyau mémorise le chemin du Python qui lance cette commande.

Attendu : message `Installed kernelspec pyds in ....` Cette opération enregistre un accès à `.venv` ; elle ne copie pas ses bibliothèques.

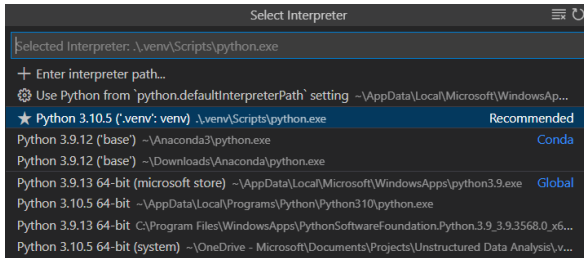
Source : IPython, Kernels for different environments.

Ouvrir le projet dans VS Code

1. Dans VS Code : **File** → **Open Folder** (Fichier → Ouvrir un dossier).
2. Sélectionner **python-data-science** dans votre dossier utilisateur, *pas* le sous-dossier `.venv`.
3. Si VS Code demande de faire confiance au dossier, confirmer seulement pour ce dossier que vous venez de créer.
4. Ouvrir **Terminal** → **New Terminal**.

Windows : flèche près du bouton **+** du terminal → **Command Prompt**. Mac : garder `zsh`. Dans ce nouveau terminal, réactiver `.venv` avec la commande de votre système.

VS Code : sélectionner l'interpréteur



1. Ouvrir la palette : Ctrl + Maj + P (Windows), Cmd + Maj + P (Mac).
2. Saisir **Python : Select Interpreter**.
3. Choisir Python 3.13 dans **.venv** du dossier du cours.

Ne pas choisir un Python global simplement parce qu'il est marqué *Recommended*.

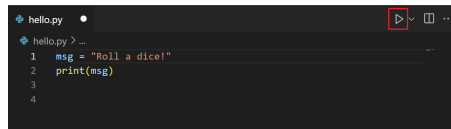
Absent de la liste ? **Enter interpreter path** : choisir le fichier Python dont le chemin a été vérifié dans le terminal.

Source : Capture officielle : VS Code, Python tutorial. Noms et versions de la capture illustratifs.

Premier script : bonjour.py

Dans l'Explorateur de VS Code, cliquer sur **New File**. Nommer le fichier bonjour.py, écrire puis enregistrer (Ctrl + S ou Cmd + S) :

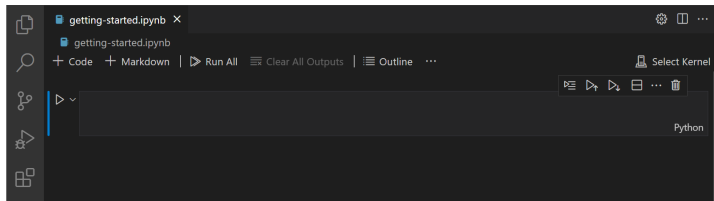
```
print("Bonjour, Data Science !")  
print(12 / (30 / 60))
```



Cliquer sur **Run Python File in Terminal**, triangle en haut à droite. Attendu dans le terminal : le message, puis 24.0.

Source : Capture officielle du bouton : VS Code, Python tutorial. Le fichier illustré diffère de notre exemple.

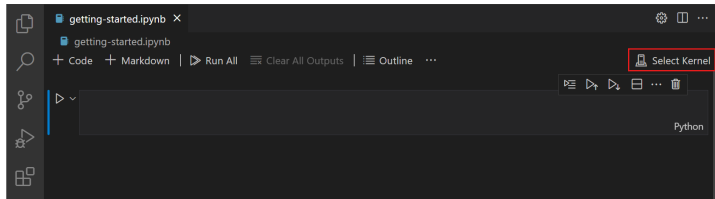
Premier notebook dans VS Code



1. Créer un nouveau fichier nommé `01_demarrage.ipynb` dans le dossier du cours.
2. L'éditeur de notebook s'ouvre ; ajouter une cellule **Code** si nécessaire.
3. En haut à droite, cliquer sur **Select Kernel** avant la première exécution.

Source : Capture officielle : VS Code, Jupyter notebooks. Une cellule contient du code ; sa sortie s'affiche en dessous.

Notebook : sélectionner le bon noyau



1. Dans **Select Kernel**, ouvrir **Select Another Kernel** si nécessaire.
2. Choisir **Python Environments**, puis le Python de `.venv`.
3. Si la liste affiche les noyaux Jupyter, choisir **Python (Data Science)**.

Le choix du noyau du notebook est distinct du choix d'interpréteur des scripts. Vérifier le chemin dans la première cellule, même si le nom paraît correct.

Source : Capture officielle : VS Code, Select a kernel. Le menu et les noms varient avec la version.

Première cellule : vérifier le Python réellement utilisé

Saisir ce code dans une cellule, puis appuyer sur **Maj + Entrée** :

```
import sys
import numpy as np

print(sys.executable)
print(np.__version__)
print(np.mean([10, 20, 30]))
```

Attendu : un chemin dans `.venv`, une version de NumPy, puis `20.0`. Le nombre de version peut varier. Si le chemin ne correspond pas à celui du terminal, changer de noyau avant de continuer.

Deuxième cellule : vérifier l'affichage d'un graphique

Ajouter une cellule **Code** sous la précédente et l'exécuter :

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [10, 20, 15], marker="o")
plt.xlabel("Jour")
plt.ylabel("Tasses de cafe")
plt.show()
```

Attendu : une ligne reliant trois points, qui monte puis redescend. Ce test vérifie l'installation ; le fonctionnement des bibliothèques sera étudié plus tard.

Jupyter Notebook dans le navigateur

Autre interface pour le *même* fichier et le *même* environnement. Dans le terminal, depuis le dossier du cours, avec `.venv` activé :

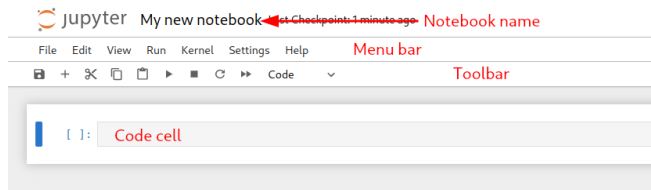
```
python -m notebook
```

1. Laisser ce terminal ouvert : il fait fonctionner le serveur Jupyter.
2. Le navigateur s'ouvre sur une adresse locale, souvent `localhost:8888`.
3. Sinon, copier dans le navigateur l'URL complète affichée par le terminal.

Le port peut changer. Ne pas partager l'URL contenant un *token* : c'est un accès à votre session. Aucun compte Jupyter en ligne n'est nécessaire.

Source : Project Jupyter, The Jupyter Notebook.

Navigateur : ouvrir le fichier et choisir son noyau

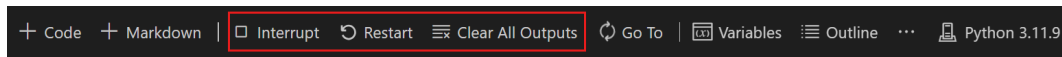


1. Dans la liste des fichiers, ouvrir `01_demarrage.ipynb`.
2. Choisir **Python (Data Science)** dans la boîte de sélection du noyau ; sinon passer par **Kernel** → **Change Kernel**.
3. Réexécuter la cellule de vérification avec **Maj + Entrée**.

Pour un nouveau fichier : **New** → **Notebook**, puis choisir le noyau. Éviter d'éditer le même fichier simultanément dans le navigateur et VS Code.

Source : Capture officielle de Notebook 7 : Project Jupyter.

Enregistrer, relancer, arrêter



1. Enregistrer le notebook avec Ctrl + S ou Cmd + S.
2. Redémarrer le noyau : **Restart**, puis **Run All** dans VS Code ; menu **Kernel** dans Jupyter.
3. Vérifier que toutes les cellules réussissent de haut en bas.
4. Pour arrêter le serveur navigateur : revenir au terminal, Ctrl + C, puis confirmer l'arrêt si demandé.

Fermer l'onglet du navigateur ne suffit pas à arrêter le serveur. Le redémarrage efface les variables en mémoire, pas le code enregistré.

Source : Capture officielle de la barre d'outils : VS Code, Jupyter notebooks.

Reprendre le travail au cours suivant

Windows : cmd

```
cd /d "%USERPROFILE%"  
cd python-data-science  
.venv\Scripts\activate.bat
```

macOS : Terminal

```
cd ~/python-data-science  
source .venv/bin/activate
```

- Ouvrir le dossier dans VS Code et vérifier le noyau du notebook.
- Ou lancer `python -m notebook` pour travailler dans le navigateur.
- Ne pas réinstaller Python et ne pas recréer `.venv` à chaque séance.

Ne pas déplacer le dossier `.venv` : ses chemins sont liés à son emplacement.

Dépannage : commandes et installation

Symptôme	Première action
SyntaxError après <code>pip install</code> <code>py</code> ou <code>python3.13</code> introuvable <code>activate.bat</code> sans effet	Vous êtes probablement dans Python : quitter <code>»»</code> avec <code>exit()</code> , puis utiliser le terminal. Rouvrir le terminal ; contrôler la réussite de l'installation et la version. Sous Windows, utiliser un terminal Command Prompt , pas PowerShell.
<code>pip</code> introuvable Erreur HTTPS/certificat	Activer <code>.venv</code> , puis utiliser <code>python -m pip</code> . Mac : vérifier Install Certificates.command . Réseau géré : contacter le support, sans désactiver TLS.

Dépannage : noyaux et bibliothèques

Symptôme	Première action
<code>ModuleNotFoundError</code>	Comparer <code>sys.executable</code> dans le notebook et le terminal ; sélectionner <code>.venv</code> , puis installer le paquet manquant dans cet environnement.
Aucun noyau proposé	Vérifier l'extension Jupyter, <code>ipykernel</code> dans <code>.venv</code> , puis rouvrir le sélecteur.
Noyau qui ne démarre plus	Le dossier a-t-il été déplacé ou supprimé ? Demander de l'aide pour recréer l'environnement et réenregistrer le noyau.
Cellule toujours occupée	Essayer Interrupt . Si nécessaire, redémarrer le noyau et relancer toutes les cellules.
Navigateur non ouvert	Lire le terminal du serveur et ouvrir son URL locale complète ; vérifier qu'il n'affiche pas une erreur.

Installation terminée : quatre preuves

1. `bonjour.py` affiche le message attendu et `24.0` dans VS Code.
2. Le notebook affiche le chemin Python du dossier `.venv`.
3. NumPy retourne `20.0` et Matplotlib affiche les trois points.
4. Après redémarrage du noyau, **Run All** reproduit les résultats sans erreur.

Question de compréhension

Où faut-il regarder si un import fonctionne dans le terminal mais échoue dans le notebook ?

Réponse : comparer les interpréteurs réellement utilisés, pas seulement leurs noms affichés.

Les opérateurs arithmétiques

Opérateur	Rôle sur des nombres	Exemple	Résultat
+	Additionner	$7 + 2$	9
-	Soustraire	$7 - 2$	5
*	Multiplier	$7 * 2$	14
/	Diviser (résultat flottant)	$7 / 2$	3.5
//	Quotient arrondi vers le bas	$7 // 2$	3
%	Reste de la division (modulo)	$7 \% 2$	1
**	Élever à une puissance	$2 ** 5$	32

Les parenthèses regroupent les calculs : $(2 + 3) * 4$ donne 20, alors que $2 + 3 * 4$ donne 14.

Avec `//`, « vers le bas » signifie vers moins l'infini : $-7 // 2$ donne -4.

Source : Python Tutorial, Numbers.

Expressions et valeurs

```
13 - 4  
(13 - 4) / (12 * 12)  
7 // 2  
7 % 2  
2 ** 5
```

Question de compréhension

Prédire la valeur et le type de chaque expression

Types élémentaires

Type	Exemple	Rôle
int, float	12, 3.5	calcul numérique
bool	True	décision
str	"Paris"	texte
NoneType	None	absence explicite

Affectation

```
distance_km = 12
duration_min = 30
speed = distance_km / (duration_min / 60)
distance_km = 15
```

Question de compréhension

Que valent `distance_km` et `speed` après chaque ligne ?

Le programme ne recalcule pas le passé

L'affectation évalue l'expression, puis associe son résultat à un nom. Changer `distance_km` plus tard ne modifie pas automatiquement `speed`.

Chaînes et indices

```
city = "Paris"  
city[0]      # "P"  
city[1:4]    # "ari"  
len(city)    # 5
```

La première position est 0. La borne droite d'une tranche est exclue.

Méthodes de chaînes

```
label = "  Data Science  "  
clean = label.strip().lower()  
clean.startswith("data")      # True  
clean.replace("science", "analyse")
```

Les chaînes sont immuables : chaque transformation retourne une nouvelle chaîne.

Découper, assembler, formater

```
parts = "Alice;42;Paris".split(";")  
name, age, city = parts  
line = " | ".join([name, city])  
message = f"{name}: {float(age):.1f}"
```

Question de compréhension

Quels objets sont des chaînes, et lequel est une liste ?

Entrée et conversion

```
age_text = input("Age: ")  
age = int(age_text)  
print(f"Dans un an: {age + 1}")
```

Question de compréhension

Pourquoi `input("Age: ") + 1` échoue-t-il ?

Débogage : valeur ou type ?

```
price = input("Prix: ")  
quantity = 3  
total = price * quantity  
print(total)
```

Si l'utilisateur saisit 12, le programme affiche 121212.

Question de compréhension

Corriger le programme et expliquer l'erreur

Comparaisons et booléens

```
temperature = 24
temperature > 20
0 <= temperature <= 40
temperature < 0 or temperature > 50
not (temperature == 24)
```

Une condition est une expression dont Python demande la valeur booléenne.

Partitionner les cas

```
if age < 0:
    price = None
elif age < 12:
    price = 0
elif student:
    price = 8
else:
    price = 12
```

Chaque visiteur doit atteindre exactement une branche.

L'ordre des branches

```
if number % 3 == 0:  
    label = "multiple de 3"  
elif number % 15 == 0:  
    label = "multiple de 15"
```

Question de compréhension

Que reçoit 30 ? Comment réordonner les cas ?

Cas limites

Pour une règle de tarif par âge, tester les valeurs juste avant, sur et juste après chaque frontière.

-1, 0, 11, 12, 17, 18

Boucle for

```
total = 0
for value in [3, 5, 2]:
    total = total + value
print(total)
```

La variable `value` reçoit successivement chaque élément.

Trace de l'accumulateur

Tour	value	total après le tour
0	—	0
1	3	3
2	5	8
3	2	10

range et borne exclue

```
for i in range(1, 4):  
    print(i)  
  
for i in range(4, 0, -1):  
    print(i)
```

Question de compréhension

Écrire range pour parcourir les entiers de 3 à 8 inclus

Trois états de boucle

```
count = 0
selected = []
highest = readings[0]
for value in readings:
    if value > 20:
        count += 1
        selected.append(value)
    if value > highest:
        highest = value
```

Compteur, collection construite et meilleur candidat.

Boucle while

```
attempts = 3
while attempts > 0:
    guess = input("Secret: ")
    if guess == "python":
        break
    attempts -= 1
```

L'état doit progresser vers l'arrêt.

Boucle infinie

```
attempts = 3
while attempts > 0:
    print("Try again")
```

Question de compréhension

Quelle variable doit changer ? Où placer la modification ?

Une fonction comme calcul nommé

```
def celsius_to_fahrenheit(celsius):  
    return celsius * 9 / 5 + 32  
  
morning = celsius_to_fahrenheit(18)
```

Entrée : celsius. Sortie : une température convertie.

return et print

```
def bad_square(x):  
    print(x * x)  
  
result = bad_square(4)  
print(result)          # None
```

Question de compréhension

Réécrire la fonction pour pouvoir réutiliser le carré

Contrat minimal

- Que représente chaque paramètre ?
- Quelle valeur revient à l'appelant ?
- Quelles entrées sont invalides ?
- Quels exemples vérifient le contrat ?

Méthode de résolution

1. Reformuler la question
2. Définir entrées et sortie
3. Calculer deux exemples et une frontière
4. Nommer l'état nécessaire
5. Écrire le pseudo-code
6. Tracer avant d'exécuter
7. Comparer obtenu et attendu

Étude guidée : bulletin météo

```
def weather_report(values, threshold):  
    total = 0.0  
    alerts = 0  
    highest = values[0]  
    for value in values:  
        total += value  
        if value > threshold:  
            alerts += 1  
        if value > highest:  
            highest = value  
    return total / len(values), alerts, highest
```

Avant l'exécution

Pour `values = [18, 21, 19, 24]` et `threshold = 20` :

- préparer les colonnes `value`, `total`, `alerts`, `highest`
- compléter quatre lignes de trace
- annoncer le tuple attendu

Travail appliqué

1. Valeur et type de huit expressions
2. Tarif de livraison avec cas invalide
3. Comptage lettres, chiffres et espaces
4. Somme d'un intervalle inclusif
5. Bulletin météo complet

Synthèse

- Une expression produit une valeur d'un type donné
- Une condition sépare les cas
- Une boucle fait évoluer un état observable
- Une fonction retourne un résultat défini par son contrat
- Un test compare obtenu et attendu

Source : Python Tutorial ; MIT 6.100L, lectures 1 à 3 et 7

CHAPITRE 2

Fonctions, collections et programmes fiables

Des fonctions que l'on peut expliquer, des données que l'on sait organiser

Le fil du chapitre

Une fois les boucles et les conditions en place, deux questions reviennent : **où ranger les données** et **comment réutiliser un calcul** ?

- Une fonction reçoit des valeurs et renvoie un résultat utilisable ailleurs.
- Une liste garde un ordre. Un tuple rassemble des positions fixes.
- Un dictionnaire associe une clé à une valeur. Un ensemble mémorise des éléments distincts.
- Une modification peut toucher plusieurs noms. Les tests rendent ces effets visibles.

Sur les diapositives « À vous », cherchez d'abord la réponse ou quelques lignes de code. La correction arrive sur la diapositive suivante.

Une fonction sépare le calcul de son utilisation

- `def` définit la fonction. Le corps s'exécute seulement lors d'un appel.
- Les paramètres nomment les valeurs reçues par cet appel.
- `return` termine l'appel et fournit son résultat. `print` affiche une valeur à l'écran.

Exemple de découpage. Le programme principal lit les données, appelle une fonction de calcul, puis affiche le résultat. On peut tester le calcul avec des valeurs écrites à la main, sans ouvrir de fichier ni demander une saisie.

Le **contrat** précise les entrées admises, le résultat attendu et les effets éventuels sur les objets reçus.

Un montant après remise

À vous. Écrivez `montant_net(montant, taux)`.

- `montant` est un nombre positif ou nul.
- `taux` est compris entre 0 et 1. Par exemple, 0,20 représente 20 %.
- La fonction renvoie le montant restant après la remise, sans l'afficher elle-même.

On suppose ici les entrées valides. Prévoyez le résultat de ces appels, puis écrivez la fonction.

```
# Appels à essayer après avoir écrit la fonction
print(montant_net(100, 0.20))
print(montant_net(50, 0))
print(montant_net(50, 1))
```

Quel test représente une remise nulle ? Une remise totale ?

Le montant restant : correction

```
def montant_net(montant, taux):  
    return montant * (1 - taux)  
  
print(montant_net(100, 0.20))  
print(montant_net(50, 0))  
print(montant_net(50, 1))
```

Résultat

```
80.0  
50  
0
```

$1 - \text{taux}$ est la part conservée. Les cas 0 et 1 vérifient les deux bornes du contrat. L'appelant peut afficher le résultat ou l'utiliser dans un autre calcul.

Les noms à l'intérieur d'une fonction

À vous. Sans exécuter le programme, prévoyez les deux lignes affichées. Le nom `prix` apparaît dans la fonction et en dehors : désigne-t-il la même variable locale ?

```
prix = 100

def reduire(prix):
    prix = prix * 0.8
    return prix

print(reduire(prix))
print(prix)
```

La portée locale : correction

```
prix = 100

def reduire(prix):
    prix = prix * 0.8
    return prix

print(reduire(prix))
print(prix)
```

Résultat

```
80.0
100
```

Le paramètre `prix` est un nom local à l'appel. L'affectation dans la fonction change ce nom local. Le `prix` du programme principal conserve 100. Une fonction peut toutefois modifier une *liste* reçue en paramètre : nous allons distinguer ces deux situations.

Un tuple pour renvoyer deux résultats

Un **tuple** regroupe des valeurs dans un ordre fixé. La virgule le construit : (2, 5) contient deux valeurs, (2,) une seule. (2) est simplement le nombre 2.

À vous. Écrivez `min_max(valeurs)` pour une liste non vide. La fonction doit renvoyer d'abord le minimum, puis le maximum. Vous pouvez utiliser `min` et `max`.

```
valeurs = [4, 9, 2, 7]
# Écrire min_max, puis récupérer les deux résultats
# dans les variables bas et haut.
```

Que signifie l'ordre des deux valeurs renvoyées ?

Le retour multiple : correction

```
def min_max(valeurs):  
    return min(valeurs), max(valeurs)  
  
bas, haut = min_max([4, 9, 2, 7])  
print(bas, haut)
```

Résultat

```
2 9
```

`return min(...), max(...)` renvoie un tuple de deux éléments. L'affectation `bas, haut = ...` le déballe dans le même ordre.

Le nombre de noms doit correspondre au nombre de valeurs. On peut aussi écrire `premier, *milieu, dernier = [10, 20, 30, 40]` : `milieu` reçoit la liste `[20, 30]`.

Chaîne, liste et tuple

Type	Ce qu'il conserve	Modification
<code>str</code>	Du texte, caractère par caractère	On construit une nouvelle chaîne.
<code>list</code>	Des éléments dans un ordre donné	On peut ajouter, retirer ou remplacer un élément.
<code>tuple</code>	Des valeurs à des positions fixes	On ne peut pas remplacer une position.

Ces séquences ont une longueur, des indices et des tranches. Un tuple peut contenir une liste : ses positions restent fixes, mais cette liste peut encore changer.

Lire une liste

À vous. Prévoyez les quatre résultats. Quelle différence faites-vous entre un indice et une tranche ?

```
scores = [12, 18, 15]
print(scores[0])
print(scores[-1])
print(scores[1:3])
print(list("data"))
```

Que se passerait-il avec `scores[3]` ? Et avec `scores[1:10]` ?

Indices et tranches : correction

```
scores = [12, 18, 15]
print(scores[0])
print(scores[-1])
print(scores[1:3])
print(list("data"))
```

Résultat

```
12
15
[18, 15]
['d', 'a', 't', 'a']
```

L'indice 0 désigne le premier élément et -1 le dernier. La borne droite d'une tranche est exclue. `scores[3]` lève `IndexError`. La tranche `scores[1:10]` s'arrête à la fin et donne `[18, 15]`.

Ajouter un élément ou plusieurs

À vous. Exécutez ces instructions une par une. Avant chaque exécution, écrivez le contenu attendu de la liste et sa longueur.

```
a = [1, 2]
a.append([3, 4])

b = [1, 2]
b.extend([3, 4])
b.insert(0, 99)
```

`append` reçoit un objet. `extend` parcourt les éléments reçus. `insert` indique une position.

Les ajouts : correction

Instruction	Contenu après l'instruction	Longueur
<code>a.append([3, 4])</code>	<code>[1, 2, [3, 4]]</code>	3
<code>b.extend([3, 4])</code>	<code>[1, 2, 3, 4]</code>	4
<code>b.insert(0, 99)</code>	<code>[99, 1, 2, 3, 4]</code>	5

```
a = [1, 2]
a.append([3, 4])
b = [1, 2]
b.extend([3, 4])
b.insert(0, 99)
```

`append` ajoute la liste comme un seul élément. `extend` ajoute ses deux éléments séparément.

Retirer une valeur ou une position

À vous. Après chaque ligne, notez ce qu'il reste dans `valeurs`. Prévoyez aussi les valeurs de dernier et premier.

```
valeurs = [10, 20, 20, 30]
dernier = valeurs.pop()
premier = valeurs.pop(0)
valeurs.remove(20)
del valeurs[:1]
```

`pop` retire à une position et renvoie l'élément retiré. `remove` cherche une valeur et retire sa première occurrence.

Les suppressions : correction

Instruction	Liste restante	Valeur renvoyée
<code>pop()</code>	<code>[10, 20, 20]</code>	30
<code>pop(0)</code>	<code>[20, 20]</code>	10
<code>remove(20)</code>	<code>[20]</code>	None
<code>del valeurs[:1]</code>	<code>[]</code>	Instruction sans résultat

Un `pop` sur une position inexistante lève `IndexError`. Un `remove` sur une valeur absente lève `ValueError`.

`valeurs.index(x)` cherche la première position de `x` et échoue si `x` est absent.

`valeurs.count(x)` compte ses occurrences, sans modifier la liste.

Ce que renvoie append

À vous. Les deux affichages montrent-ils la même chose ?

```
scores = [12, 18]
resultat = scores.append(15)
print(resultat)
print(scores)
```

Pourquoi la ligne `scores = scores.append(15)` poserait-elle problème ?

Modifier un objet et renvoyer une valeur

```
scores = [12, 18]
resultat = scores.append(15)
print(resultat)
print(scores)
```

Résultat

```
None
[12, 18, 15]
```

`append` modifie la liste et renvoie `None`. Dans `scores = scores.append(15)`, l'ajout a lieu, puis le nom `scores` reçoit `None`.

`extend`, `insert`, `remove`, `sort` et `reverse` renvoient aussi `None`. `pop` est différent : il renvoie l'élément retiré.

Trier en conservant la liste de départ

À vous. Prévoyez les trois affichages.

```
notes = [12, 18, 15]
ordre = sorted(notes)
print(notes, ordre)
retour = notes.sort()
print(notes)
print(retour)
```

Quelle opération choisir si l'on veut garder les notes dans leur ordre initial ?

sort et sorted : correction

```
notes = [12, 18, 15]
ordre = sorted(notes)
print(notes, ordre)
retour = notes.sort()
print(notes)
print(retour)
```

Résultat

```
[12, 18, 15] [12, 15, 18]
[12, 15, 18]
None
```

`sorted` construit une nouvelle liste. `sort` trie la liste existante. L'option `reverse=True` demande un tri décroissant. La méthode `reverse()` inverse seulement l'ordre actuel, sans trier les valeurs.

Deux noms pour une même liste

À vous. Dessinez les listes et les noms qui les désignent, puis prévoyez les trois affichages.

```
a = [10, 20]
b = a
c = a.copy()
b.append(30)
c[0] = 99
print(a)
print(b)
print(c)
```

À quelle ligne crée-t-on une nouvelle liste ?

Alias et copie : correction

```
a = [10, 20]
b = a
c = a.copy()
b.append(30)
c[0] = 99
print(a)
print(b)
print(c)
```

Résultat

```
[10, 20, 30]
[10, 20, 30]
[99, 20]
```

`b = a` crée un autre nom pour la même liste. `a.copy()` crée une nouvelle liste. Un paramètre de fonction peut lui aussi désigner la liste de l'appelant : un `append` dans la fonction serait visible après l'appel.

Une copie peut encore partager des objets

À vous. Ici, chaque élément de la liste est lui-même une liste. Que vont afficher les deux `print` ?

```
original = [[1, 2], [3, 4]]
copie = original.copy()
copie[0].append(99)
print(original)
print(copie)
```

La copie concerne-t-elle la liste extérieure, les listes intérieures ou les deux ?

La copie superficielle : correction

```
original = [[1, 2], [3, 4]]
copie = original.copy()
copie[0].append(99)
print(original)
print(copie)
```

Résultat

```
[[1, 2, 99], [3, 4]]
[[1, 2, 99], [3, 4]]
```

Les deux listes extérieures sont distinctes. Leurs éléments désignent encore les mêmes listes intérieures. `copy()` et la tranche `[:]` font toutes deux une copie superficielle. Ajouter un élément à `copie` seule ne l'ajouterait pas à `original`.

Un numéro avec chaque valeur

À vous. Affichez les noms avec une numérotation qui commence à 1. `enumerate(noms, start=1)` fournit à chaque tour un couple numéro-valeur.

```
noms = ["Ada", "Linus", "Guido"]  
# Écrire une boucle qui affiche le numéro et le nom.
```

Pourquoi n'a-t-on pas besoin d'incrémenter un compteur à la main ?

Le parcours avec enumerate : correction

```
noms = ["Ada", "Linus", "Guido"]  
for numero, nom in enumerate(noms, start=1):  
    print(numero, nom)
```

Résultat

```
1 Ada  
2 Linus  
3 Guido
```

enumerate avance le compteur en même temps que le parcours. Le déballage attribue les deux valeurs à `numero` et `nom`. Avec `start=1`, on obtient des numéros de présentation, pas les indices Python de la liste.

Associer des noms et des notes

À vous. `zip(noms, notes)` associe les éléments situés aux mêmes positions. Écrivez une boucle qui affiche chaque nom avec sa note. Prévoyez ce qui arrivera à Guido.

```
noms = ["Ada", "Linus", "Guido"]
notes = [18, 15]
# Parcourir zip(noms, notes) et afficher chaque couple.
```

zip s'arrête au plus court : correction

```
noms = ["Ada", "Linus", "Guido"]  
notes = [18, 15]  
for nom, note in zip(noms, notes):  
    print(nom, note)
```

Résultat

```
Ada 18  
Linus 15
```

Guido n'a pas de note correspondante. Le parcours s'arrête après deux couples. Si les longueurs doivent être égales, `zip(noms, notes, strict=True)` lève `ValueError` lorsque l'écart apparaît pendant le parcours (Python 3.10 ou plus récent).

Nettoyer une liste en gardant son ordre

À vous. Construisez une nouvelle liste qui retire les espaces aux extrémités de chaque nom et ignore les noms devenus vides. Gardez l'ordre d'arrivée.

```
noms = [" Ada ", "", " Linus", "   "]
propres = []
# Parcourir les noms, nettoyer, puis ajouter si non vide.
```

`texte.strip()` renvoie le texte sans espaces aux extrémités. Commencez par une boucle ordinaire. Que doit contenir `propres` après chaque tour ?

Le nettoyage par une boucle : correction

```
noms = [" Ada ", "", " Linus", "   "]
propres = []
for nom in noms:
    nettoye = nom.strip()
    if nettoye != "":
        propres.append(nettoye)
print(propres)
```

Résultat

```
['Ada', 'Linus']
```

Après chaque tour, propres contient les noms non vides déjà lus, nettoyés et dans le même ordre. On laisse la liste parcourue intacte. Supprimer ses éléments pendant le parcours risquerait de décaler les indices et de sauter un élément.

La même transformation en compréhension

Une compréhension construit une liste avec une expression de transformation, un parcours et éventuellement un filtre.

À vous. Réécrivez le nettoyage en une compréhension. Le test doit porter sur le nom *après* retrait des espaces.

```
noms = [" Ada ", "", " Linus", "   "]  
# propres = [expression for nom in noms if condition]
```

Le résultat doit rester celui de la boucle précédente. La liste de départ doit rester inchangée.

La compréhension : correction

```
noms = [" Ada ", "", " Linus", "   "]
propres = [nom.strip() for nom in noms if nom.strip() != ""]
print(propres)
```

Résultat

```
['Ada', 'Linus']
```

- `nom.strip()` est la valeur ajoutée à la nouvelle liste.
- `for nom in noms` définit le parcours.
- `if nom.strip() != ""` sélectionne les valeurs à garder.

La boucle précédente évite de calculer `strip()` deux fois et se complète plus facilement si plusieurs règles s'ajoutent. Les deux écritures conviennent ici.

Un dictionnaire associe une clé à une valeur

Un dictionnaire permet de nommer les informations plutôt que de mémoriser leur position.

```
client = {"id": 17, "ville": "Lille"}
```

- "ville" est une clé. "Lille" est sa valeur.
- `client[cle]` lit la valeur associée à la clé.
- `client[cle] = valeur` ajoute la clé ou remplace sa valeur si elle existe déjà.
- Une clé n'apparaît qu'une fois. Plusieurs clés peuvent avoir la même valeur.

Un dictionnaire peut représenter un client, un compteur de mots ou un total par catégorie.

Une clé présente, une clé absente

À vous. Prévoyez les résultats, puis ajoutez "segment": "retail" au dictionnaire.

```
client = {"id": 17, "ville": "Lille"}  
print(client["ville"])  
print(client.get("pays", "FR"))  
# Ajouter le segment, puis afficher client.
```

get(cle, default) renvoie la valeur par défaut si la clé manque. Que ferait client["pays"] ?
Le get précédent crée-t-il cette clé ?

Lire et compléter le dictionnaire : correction

```
client = {"id": 17, "ville": "Lille"}  
print(client["ville"])  
print(client.get("pays", "FR"))  
client["segment"] = "retail"  
print(client)
```

Résultat

```
Lille  
FR  
{'id': 17, 'ville': 'Lille', 'segment': 'retail'}
```

`client["pays"]` lève `KeyError`. `get` fournit une valeur de remplacement sans ajouter la clé. On utilise les crochets quand la présence est requise, et `get` quand l'absence est prévue.

Parcourir un dictionnaire qui évolue

À vous. Prévoyez le contenu de `cles` et de `instantane` après l'ajout. Écrivez ensuite une boucle qui affiche chaque clé avec sa valeur.

```
client = {"id": 17, "ville": "Lille"}
cles = client.keys()
instantane = list(client.keys())
client["pays"] = "FR"
# Afficher list(cles), instantane, puis parcourir client.items().
```

`keys()` donne les clés, `values()` les valeurs et `items()` les couples clé-valeur.

Une vue suit les changements : correction

```
client = {"id": 17, "ville": "Lille"}
cles = client.keys()
instantane = list(client.keys())
client["pays"] = "FR"
print(list(cles))
print(instantane)
for cle, valeur in client.items():
    print(cle, valeur)
```

Résultat

```
['id', 'ville', 'pays']
['id', 'ville']
id 17
ville Lille
pays FR
```

keys, values et items donnent des vues liées au dictionnaire. La liste instantane garde les clés présentes au moment de sa création. L'itération suit l'ordre d'insertion des clés.

Compter les mots d'une phrase

À vous. Produisez un dictionnaire qui associe chaque mot à son nombre d'occurrences.

- Passez le texte en minuscules, puis séparez-le avec `split()`.
- Retirez `.,!?:` aux extrémités de chaque morceau avec `strip(".,!?:")`.
- Ignorez les morceaux devenus vides. Les autres caractères restent tels quels.

```
texte = "Chat chat, chien."  
comptes = {}  
# Pour chaque mot nettoyé, augmenter son compteur.
```

Comment initialiser le compteur d'un mot que l'on rencontre pour la première fois ?

Le dictionnaire de fréquences : correction

```
texte = "Chat chat, chien."  
comptes = {}  
for morceau in texte.lower().split():  
    mot = morceau.strip(".,!?;:")  
    if mot != "":  
        comptes[mot] = comptes.get(mot, 0) + 1  
print(comptes)
```

Résultat

```
{'chat': 2, 'chien': 1}
```

Les états successifs sont {'chat': 1}, puis {'chat': 2}, puis {'chat': 2, 'chien': 1}.
get(mot, 0) fournit 0 au premier passage. Après chaque tour, le dictionnaire compte les mots déjà traités.

Un ensemble pour mémoriser les valeurs déjà vues

Un **ensemble** contient des éléments distincts, sans ordre de parcours garanti. `set()` crée un ensemble vide. `x in vus` teste la présence et `vus.add(x)` ajoute un élément.

À vous. Gardez la première occurrence de chaque identifiant, dans l'ordre d'arrivée. Utilisez une liste de résultat et un ensemble des valeurs vues.

```
identifiants = ["A", "B", "A", "C", "B"]
vus = set()
uniques = []
# Ajouter à uniques seulement les identifiants encore inconnus.
```

Quel rôle joue chacune des deux collections ?

La déduplication stable : correction

```
identifiants = ["A", "B", "A", "C", "B"]
vus = set()
uniques = []
for identifiant in identifiants:
    if identifiant not in vus:
        uniques.append(identifiant)
        vus.add(identifiant)
print(uniques)
```

Résultat

```
['A', 'B', 'C']
```

La liste passe par ["A"], ["A", "B"], reste identique au troisième tour, puis devient ["A", "B", "C"]. Le dernier B ne change rien. La liste conserve l'ordre. L'ensemble répond à « déjà vu ? ».

Comparer deux ensembles

À vous. Quels identifiants appartiennent à chacun des quatre résultats ? L'ordre dans lequel vous les écrivez n'a pas d'importance.

```
actifs = {"A", "B", "C"}  
contactes = {"B", "D"}  
# actifs & contactes  
# actifs | contactes  
# actifs - contactes  
# actifs ^ contactes
```

& signifie « dans les deux », | « dans au moins un », - « dans le premier seulement » et ^ « dans un seul des deux ».

Les opérations sur les ensembles : correction

Expression	Éléments du résultat
<code>actifs & contacts</code>	<code>{"B"}</code>
<code>actifs contacts</code>	<code>{"A", "B", "C", "D"}</code>
<code>actifs - contacts</code>	<code>{"A", "C"}</code>
<code>actifs ^ contacts</code>	<code>{"A", "C", "D"}</code>

`add(x)` ajoute un élément et `update(elements)` en ajoute plusieurs. `remove(x)` lève

`KeyError` si l'élément manque. `discard(x)` tolère cette absence.

Un ensemble ne possède pas d'indice comme `vus[0]`. Pour un affichage reproductible de ces chaînes, on peut afficher `sorted(vus)`, qui construit une liste triée.

Quelles valeurs peuvent servir de clés ?

Les clés d'un dictionnaire et les éléments d'un ensemble doivent être **hachables**. Python doit pouvoir leur associer une valeur de hachage stable pendant leur vie.

À vous. Parmi ces objets, lesquels peuvent être des clés de dictionnaire ?

```
17  
"Lille"  
(48.8, 2.3)  
[48.8, 2.3]  
("point", [48.8, 2.3])
```

Le fait que le dernier objet soit un tuple suffit-il ?

Les clés hachables : correction

Objet	Clé possible ?	Pourquoi ?
17	Oui	Un entier est hachable.
"Lille"	Oui	Une chaîne est hachable.
(48.8, 2.3)	Oui	Tous les éléments du tuple sont hachables.
[48.8, 2.3]	Non	Une liste est mutable et non hachable.
("point", [48.8, 2.3])	Non	Le tuple contient une liste non hachable.

L'immutabilité des positions du tuple ne rend pas ses éléments immuables. Une clé non hachable provoque `TypeError`. Pour les types usuels de ce chapitre, les nombres, les chaînes et les tuples de valeurs hachables conviennent.

Un tri à partir d'un champ

À vous. Triez les lignes par délai croissant et gardez lignes dans son ordre initial. L'argument `key` reçoit une fonction qui calcule la valeur à comparer pour chaque ligne.

```
lignes = [{"ville": "Lyon", "delai": 12},  
          {"ville": "Paris", "delai": 5}]  
# Créer ordre avec sorted et une clé de tri.
```

`lambda ligne: ligne["delai"]` est une petite fonction sans nom. Elle reçoit une ligne et renvoie son délai. Quelle ville doit apparaître en premier ?

La clé de tri : correction

```
lignes = [{"ville": "Lyon", "delai": 12},  
          {"ville": "Paris", "delai": 5}]  
ordre = sorted(lignes, key=lambda ligne: ligne["delai"])  
print([ligne["ville"] for ligne in ordre])  
print([ligne["ville"] for ligne in lignes])
```

Résultat

```
['Paris', 'Lyon']  
['Lyon', 'Paris']
```

La fonction de clé fournit 12 pour Lyon et 5 pour Paris. `sorted` classe selon ces nombres. `lignes.sort(key=lambda ligne: ligne["ville"])` trierait la liste existante selon les noms de villes. Les dictionnaires à l'intérieur restent les mêmes objets : ce tri ne les copie pas en profondeur.

Une erreur fait partie du contrat

Une fonction doit distinguer un résultat valide d'une entrée qu'elle ne peut pas traiter.

- `raise ValueError("message")` signale une valeur non admise et interrompt le chemin en cours.
- `try` délimite les instructions dont on veut traiter une erreur.
- `except ValueError` traite cette catégorie précise d'erreur.
- Sans traitement, l'exception remonte à l'appelant.

Par exemple, `int("sept")` échoue pendant la conversion. `int("-2")` réussit, mais -2 reste invalide si le contrat exige une quantité strictement positive.

Il faut donc vérifier à la fois la conversion et le domaine de la valeur obtenue.

Lire une quantité strictement positive

À vous. Écrivez `parse_positive_int(texte)`.

- Convertir `texte` avec `int` et renvoyer l'entier s'il est strictement positif.
- Si la conversion échoue, lever `ValueError("entier attendu")`.
- Si l'entier est nul ou négatif, lever `ValueError("valeur positive attendue")`.

```
# Cas à essayer après avoir écrit la fonction  
# "7", "0", "-2", "sept"
```

Prévoyez pour chaque entrée une valeur de retour ou le message d'erreur. Les appels invalides se testent séparément.

La conversion contrôlée : correction

```
def parse_positive_int(texte):  
    try:  
        valeur = int(texte)  
    except ValueError as erreur:  
        raise ValueError("entier attendu") from erreur  
    if valeur <= 0:  
        raise ValueError("valeur positive attendue")  
    return valeur
```

Entrée	Résultat
"7"	Retourne 7
"0" ou "-2"	ValueError : valeur positive attendue
"sept"	ValueError : entier attendu

`from erreur` conserve le lien avec l'échec de conversion initial. On intercepte seulement l'erreur que l'on sait expliquer ici.

Des tests choisis à partir du contrat

À vous. Pour la fonction précédente, trouvez un cas normal, une frontière et des cas invalides. Le nombre 1 mérite-t-il un test ? Pourquoi ?

`assert condition` ne fait rien si la condition est vraie. Sinon, il lève `AssertionError`.

```
# Reprendre la fonction parse_positive_int.  
# Ajouter les assertions pour "7" et "1".  
# Tester aussi "0", "-2" et "sept".
```

Une entrée invalide doit provoquer `ValueError`. Un résultat renvoyé normalement serait un échec du test. Les assertions servent ici à vérifier le code, pas à remplacer les validations de la fonction.

Vérifier aussi les erreurs : correction

La fonction est celle de la correction précédente.

```
assert parse_positive_int("7") == 7
assert parse_positive_int("1") == 1
for texte in ["0", "-2", "sept"]:
    try:
        parse_positive_int(texte)
    except ValueError:
        print(texte, ": refus attendu")
    else:
        raise AssertionError("refus attendu")
```

Résultat

```
0 : refus attendu
-2 : refus attendu
sept : refus attendu
```

Le else du try s'exécute seulement si aucune exception n'a eu lieu. Il détecte ici un faux succès. Le cas 1 vérifie le plus petit entier admis.

Un petit lot de commandes

On veut signaler les identifiants répétés et calculer le montant par client. **Règle de cet exercice** : on signale les doublons mais on conserve leurs montants. Les commandes annulées participent au contrôle des identifiants, mais pas aux totaux.

```
lignes = [  
    {"id": "01", "client": "Ada", "montant": 10, "statut": "ok"},  
    {"id": "02", "client": "Linus", "montant": 20, "statut": "annule"},  
    {"id": "01", "client": "Ada", "montant": 5, "statut": "ok"},  
    {"id": "03", "client": "Linus", "montant": 7, "statut": "ok"},  
]
```

Les champs sont présents et les montants sont valides. Chaque ligne décrit une observation à traiter.

Commandes : repérer les identifiants répétés

À vous. Reprenez les quatre lignes précédentes.

- Gardez dans un ensemble les identifiants déjà rencontrés.
- Lorsqu'un identifiant revient, ajoutez-le à une liste doublons.
- Conservez l'ordre de détection. Un identifiant qui revient trois fois sera signalé deux fois.

```
vus = set()
doublons = []
# Parcourir lignes et examiner ligne["id"].
```

Avant de coder, suivez l'état de `vus` et `doublons` sur les quatre lignes.

Les doublons : correction

Les données sont celles du petit lot de commandes.

```
vus = set()
doublons = []
for ligne in lignes:
    identifiant = ligne["id"]
    if identifiant in vus:
        doublons.append(identifiant)
    else:
        vus.add(identifiant)
print(doublons)
```

Résultat

```
['01']
```

Après les deux premières lignes, `vus` contient O1 et O2. La troisième ligne retrouve O1 et l'ajoute à `doublons`. La quatrième ajoute O3 à `vus`. Le statut n'intervient pas dans ce contrôle.

Commandes : calculer le montant par client

À vous. À partir du même lot, construisez totaux.

- Ignorez les lignes dont le statut vaut "annule".
- Utilisez le nom du client comme clé.
- Additionnez les montants des lignes conservées, y compris celles dont l'identifiant revient.

```
totaux = {}  
# Parcourir lignes, filtrer le statut, puis cumuler le montant.
```

Calculez d'abord à la main le total d'Ada et celui de Linus.

Les totaux : correction

Les données sont celles du petit lot de commandes.

```
totaux = {}
for ligne in lignes:
    if ligne["statut"] != "annule":
        client = ligne["client"]
        totaux[client] = totaux.get(client, 0) + ligne["montant"]
print(totaux)
```

Résultat

```
{'Ada': 15, 'Linus': 7}
```

Ada reçoit 10 puis 5. La commande de 20 pour Linus est annulée. Seule sa commande de 7 contribue au total. Le dictionnaire garde, pour chaque client, la somme des montants admis déjà lus.

Les deux calculs dans une fonction

La boucle met à jour les trois collections, puis la fonction renvoie un tuple.

```
def consolider(lignes):
    totaux, vus, doublons = {}, set(), []
    for ligne in lignes:
        identifiant = ligne["id"]
        if identifiant in vus:
            doublons.append(identifiant)
        else:
            vus.add(identifiant)
        if ligne["statut"] != "annule":
            client = ligne["client"]
            totaux[client] = totaux.get(client, 0) + ligne["montant"]
    return totaux, doublons
```

`consolider(lignes)` donne `({'Ada': 15, 'Linus': 7}, ['01'])`. `consolider([])` donne `({}, [])`. Les doublons sont signalés, pas éliminés : les retirer changerait le contrat et les totaux.

Le choix d'une collection

À vous. Associez chaque besoin à une structure et à l'opération qui vous intéresse.

1. Conserver les dossiers dans leur ordre d'arrivée.
2. Répondre souvent à « cet identifiant est-il déjà présent ? ».
3. Retrouver le total d'un client à partir de son identifiant.
4. Renvoyer des coordonnées dont les positions ne changent pas.

Pourquoi la recherche d'une valeur dans une liste devient-elle plus coûteuse quand la liste grandit ?

Le sens des données guide le choix

Besoin	Structure	Opération utile
Ordre d'arrivée	<code>list</code>	<code>append</code> , parcours dans l'ordre
Présence d'un identifiant	<code>set</code>	<code>identifiant in vus</code>
Total d'un client	<code>dict</code>	<code>totaux[client]</code>
Coordonnées fixes	<code>tuple</code>	Déballage des positions

L'accès à un indice de liste est direct. Chercher une valeur peut exiger de lire toute la liste. La recherche dans un ensemble ou par clé de dictionnaire est rapide en moyenne grâce au hachage, mais ces collections occupent aussi de la mémoire. Le chapitre 3 permettra de quantifier ces coûts.

Application : un index par identifiant

À vous. Écrivez `indexer(clients)` qui renvoie un dictionnaire associant chaque identifiant à sa position dans la liste.

- Les identifiants sont des chaînes et le champ "id" est toujours présent.
- Une liste vide doit donner un dictionnaire vide.
- Un identifiant répété doit provoquer `ValueError`, sans écrasement silencieux.

```
clients = [{"id": "C2", "ville": "Lyon"},  
           {"id": "C1", "ville": "Paris"}]  
# Écrire indexer avec enumerate et un dictionnaire.
```

Quels tests permettent de vérifier les trois règles du contrat ?

L'index secondaire : correction

```
def indexer(clients):  
    index = {}  
    for position, client in enumerate(clients):  
        identifiant = client["id"]  
        if identifiant in index:  
            raise ValueError("identifiant dupliqué")  
        index[identifiant] = position  
    return index
```

Entrée	Résultat
[{"id": "C2"}, {"id": "C1"}]	{"C2": 0, "C1": 1}
[]	{}
[{"id": "C2"}, {"id": "C2"}]	ValueError

Sans le test de présence, la seconde affectation remplacerait la première position. L'index reste valable tant que l'ordre et les identifiants de la liste ne changent pas.

Quelques méthodes à retrouver facilement

Structure	Opération	Effet
Liste	<code>a[1:3] = [7, 8]</code>	Remplace une tranche. La longueur peut changer.
Liste	<code>a.copy()</code>	Nouvelle liste extérieure, éléments partagés.
Dictionnaire	<code>d.update(autre)</code>	Ajoute des clés et remplace les valeurs des clés déjà présentes.
Dictionnaire	<code>d.pop(cle, default)</code>	Retire et renvoie la valeur, ou renvoie le défaut si la clé manque.
Ensemble	<code>s.update(elements)</code>	Ajoute plusieurs éléments distincts.

Sur un dictionnaire, `cle in d` cherche parmi les clés. Pour chercher parmi les valeurs, il faut le préciser avec `valeur in d.values()`.

Ce qu'il faut pouvoir expliquer

- Une fonction a des entrées, un résultat et un contrat. Ses noms locaux ne remplacent pas ceux de l'appelant.
- Une liste reçue en paramètre peut être modifiée. Un alias voit cette modification. Une copie superficielle partage encore les objets intérieurs.
- Une liste conserve un ordre, un ensemble des éléments distincts, un dictionnaire des associations. Un tuple fixe des positions.
- Une boucle construit un état que l'on peut suivre après chaque tour. Une compréhension convient quand la transformation reste simple.
- Les tests vérifient les résultats ordinaires, les frontières et les erreurs annoncées par le contrat.

Source : Notes du cours, chapitre 2. Python : structures de données, fonctions intégrées, exceptions.

CHAPITRE 3

Algorithmes élémentaires et complexité

Évaluer le temps, la mémoire et les compromis

Correction avant efficacité

Un programme rapide qui répond à une autre question reste faux.

1. Établir le contrat
2. Vérifier la correction
3. Analyser les coûts
4. Optimiser le goulot mesuré ou démontré

Trois façons d'évaluer

- Chronométrer : concret, mais dépend du contexte
- Compter des opérations : explique une implémentation
- Étudier l'ordre de croissance : compare les conceptions

Taille de l'entrée

- liste : $n = \text{len}(L)$
- deux listes : n et m
- matrice : r lignes et c colonnes
- journal : nombre d'événements et nombre de clés distinctes

Question de compréhension

Quelle taille décrit une chaîne contenant de très grands nombres ?

Temps et espace auxiliaire

Temps : opérations dominantes lorsque

Espace auxiliaire : mémoire supplémentaire au pic d'exécution

La sortie obligatoire doit être distinguée d'un index créé pour accélérer le calcul.

Classes de croissance

Classe	Exemple	Si n double
$O(1)$	accès direct	similaire
$O(\log n)$	recherche binaire	environ +1 étape
$O(n)$	parcours	double
$O(n \log n)$	tri	un peu plus que double
$O(n^2)$	toutes les paires	quadruple

Blocs successifs

```
for x in left:  
    process(x)  
for y in right:  
    process(y)
```

Temps $O(n + m)$. Aucun motif n'autorise à remplacer m par n .

Boucles imbriquées

```
for i in range(n):  
    for j in range(n):  
        compare(i, j)
```

n répétitions de n opérations donnent $O(n^2)$.

Attention aux boucles imbriquées

Une boucle interne ne rend pas automatiquement le coût quadratique. Si un pointeur avance au total au plus n fois sur toute l'exécution, les deux pointeurs peuvent coûter $O(n)$.

Coûts de listes

Opération	Temps usuel
$L[i]$, $\text{len}(L)$	$O(1)$
$x \text{ in } L$	$O(n)$
$L.\text{append}(x)$	$O(1)$ amorti
$L.\text{insert}(0,x)$	$O(n)$
$\text{sorted}(L)$	$O(n \log n)$

Hachage

Opération	Temps moyen
<code>k in dictionary</code>	$O(1)$
<code>x in set</code>	$O(1)$
construction avec n valeurs	$O(n)$
mémoire de l'index	$O(n)$

structure supplémentaire.

Le gain de temps se paie par une

Recherche linéaire

- aucune précondition d'ordre
- meilleur cas : première position
- pire cas : toute la liste
- temps $O(n)$, espace $O(1)$

Recherche binaire

```
while left <= right:
    middle = (left + right) // 2
    if values[middle] == target:
        return middle
    if values[middle] < target:
        left = middle + 1
    else:
        right = middle - 1
```

Invariant de recherche

Si la cible existe, elle reste dans l'intervalle $[left, right]$. Chaque tour élimine au moins la moitié des positions encore possibles.

Question de compréhension

Pourquoi la liste doit-elle être triée ?

Trier comme prétraitement

Pour q recherches :

- scans répétés : $O(qn)$
- tri puis dichotomie : $O(n \log n + q \log n)$
- ensemble : $O(n + q)$ moyen, avec mémoire $O(n)$

Le choix dépend aussi des requêtes d'ordre et de la mémoire.

Doublon : force brute

```
for i in range(len(values)):
    for j in range(i + 1, len(values)):
        if values[i] == values[j]:
            return True
return False
```

Temps $O(n^2)$, espace auxiliaire $O(1)$.

Doublon : ensemble

```
seen = set()
for value in values:
    if value in seen:
        return True
    seen.add(value)
return False
```

Temps moyen $O(n)$, espace $O(n)$.

Le goulot supprimé

La force brute répète une recherche parmi les valeurs précédentes. L'ensemble remplace cette recherche linéaire par une appartenance moyenne constante.

Deux valeurs pour une cible

```
positions = {}  
for index, value in enumerate(values):  
    complement = target - value  
    if complement in positions:  
        return positions[complement], index  
    positions[value] = index
```

Un index répond à une question plus précise que « chercher n'importe quelle valeur ».

Fenêtre glissante

Deux fenêtres voisines de largeur k partagent $k - 1$ valeurs.

- recalcul de chaque somme : $O(nk)$
- somme mise à jour : $O(n)$
- espace auxiliaire hors sortie : $O(1)$

Diagnostic d'un goulot

1. Définir les tailles
2. Repérer le travail répété
3. Calculer temps et pic mémoire
4. Proposer index, tri unique ou résumé
5. Vérifier que le contrat reste identique

Audit temps–espace

Pour chaque solution, séparer explicitement :

- les opérations répétées selon n ;
- les structures auxiliaires qui croissent selon n ;
- la sortie obligatoire, distincte de l'espace auxiliaire ;
- le pire cas et les hypothèses du cas moyen.

Du symbole à l'ordre de grandeur

Avec $n = 100\,000$, comparer approximativement n , $n \log_2 n$ et n^2 . Une constante compte en pratique, mais elle ne transforme pas une croissance quadratique en solution scalable.

Question de compréhension

Quelle expression devient le goulot lorsque n est multiplié par 10 ?

Journal d'événements

```
for position, (user_id, action) in enumerate(events):  
    users.add(user_id)  
    counts[action] = counts.get(action, 0) + 1  
    if action == "purchase" and user_id not in first_purchase:  
        first_purchase[user_id] = position
```

Un parcours, trois index adaptés aux questions.

Travail appliqué

1. Contains Duplicate : paires, tri, ensemble
2. Deux prix pour un budget
3. Moyennes mobiles
4. Stratégie pour dix mille requêtes d'appartenance

Pour chaque solution : temps, espace, hypothèses et goulot.

Synthèse

Une optimisation relie un coût dominant à une modification précise. Le hachage échange de la mémoire contre du temps ; l'ordre autorise la dichotomie ; la fenêtre réutilise le travail commun.

Source : MIT 6.100L, lectures 21 à 24 et 26 ; Fluent Python, chapitres 2 et 3

CHAPITRE 4

Conception objet pour les pipelines

Encapsuler un état et composer des étapes

Quand une classe aide

Une classe devient utile lorsqu'un concept possède :

- un état qui doit rester valide
- plusieurs opérations liées à cet état
- une interface utilisée par d'autres composants

Une transformation sans état reste souvent une fonction.

Objet comme abstraction

Représentation interne : quelles données décrivent l'objet ?

Interface : quelles opérations l'appelant peut-il demander ?

Classe et instance

- La classe définit un nouveau type
- Une instance possède ses propres valeurs d'attributs
- Plusieurs instances suivent le même contrat
- `self` désigne l'instance qui reçoit l'appel

Première classe

```
class TemperatureScale:
    def __init__(self, offset, factor):
        self.offset = offset
        self.factor = factor

    def convert(self, value):
        return value * self.factor + self.offset
```

Deux instances

```
c_to_f = TemperatureScale(32, 9 / 5)
km_to_miles = TemperatureScale(0, 0.621371)

c_to_f.convert(20)
km_to_miles.convert(10)
```

Chaque instance conserve sa configuration.

Invariant d'objet

Une propriété doit rester vraie après la construction et après chaque méthode. Exemples :
solde non négatif, début inférieur à fin, noms de colonnes uniques.

Intervalle valide

```
class Interval:
    def __init__(self, start, end):
        if start > end:
            raise ValueError("start must be <= end")
        self.start = start
        self.end = end

    def duration(self):
        return self.end - self.start
```

Dataclass

Une dataclass convient à un objet surtout décrit par ses champs.

- constructeur généré
- représentation lisible
- égalité par valeurs
- annotations visibles

Objet valeur

```
from dataclasses import dataclass

@dataclass(frozen=True)
class RecordIssue:
    row: int
    column: str
    message: str
```

`frozen=True` exprime l'intention d'une valeur immuable.

Valeur mutable par instance

```
from dataclasses import dataclass, field

@dataclass
class ValidationResult:
    valid_rows: list = field(default_factory=list)
    issues: list = field(default_factory=list)
```

Question de compréhension

Pourquoi ne pas écrire directement `issues: list = []` ?

Composition

Un pipeline contient des étapes indépendantes.

- chaque étape possède une petite interface
- le pipeline contrôle l'ordre
- chaque composant se teste seul
- une étape peut être remplacée sans modifier les autres

Pipeline composé

```
class Pipeline:
    def __init__(self, steps):
        self.steps = list(steps)

    def run(self, rows):
        current = rows
        for step in self.steps:
            current = step.transform(current)
        return current
```

Polymorphisme Python

Le pipeline ne demande pas la classe concrète. Il demande un comportement compatible : une méthode `transform`. Une classe abstraite n'est utile que si le contrat partagé exige une formalisation.

Fonction ou objet étape ?

Fonction	Objet
transformation sans état	configuration persistante
entrée vers sortie	statistiques apprises
test direct	plusieurs opérations liées

Effets aux frontières

- lire et écrire des fichiers
- consulter l'heure ou le réseau
- journaliser et afficher

Le coeur reçoit des données en mémoire et retourne des données.

Chargeur CSV

```
class CsvLoader:
    def load(self, path):
        with Path(path).open(encoding="utf-8", newline="") as stream:
            return list(csv.DictReader(stream))
```

Le gestionnaire de contexte ferme le fichier, même en cas d'erreur.

Transformation pure

```
def normalize_rows(rows):  
    return [  
        {key.strip().lower(): value.strip()  
         for key, value in row.items()}  
        for row in rows  
    ]
```

Cette fonction se teste sans système de fichiers.

Pipeline de sinistres

1. Le chargeur retourne des lignes
2. Le normaliseur standardise les champs
3. Le validateur produit lignes valides et problèmes
4. L'orchestrateur assemble et journalise

Validation observable

```
for index, row in enumerate(rows, start=2):
    missing = required - row.keys()
    if missing:
        issues.append(RecordIssue(
            index, "*", f"missing {sorted(missing)}")
        ))
    continue
```

L'erreur devient une donnée structurée.

Tester le comportement

- construire une instance valide
- refuser un état invalide
- vérifier une transition d'état
- comparer la sortie publique
- éviter de tester des détails internes

Pièges de conception

- classe qui ne contient qu'une fonction
- hiérarchie profonde pour deux comportements
- lecture de fichier cachée dans un calcul
- attribut mutable partagé entre instances
- méthode qui imprime au lieu de retourner

Fonction, dataclass ou classe ?

- calcul sans mémoire : fonction ;
- donnée structurée avec invariants simples : dataclass ;
- état et comportement évoluant ensemble : classe ;
- comportements interchangeables : composition derrière une interface.

Question de compréhension

Une classe apporte-t-elle ici une abstraction, ou seulement du bruit ?

Revue d'une API de pipeline

Une bonne frontière permet de répondre :

1. quelles entrées sont acceptées ?
2. quelles erreurs sont prévisibles ?
3. quel état est mémorisé ?
4. que peut-on tester sans fichier ni réseau ?

Travail appliqué

1. Compte bancaire et invariants
2. Dataclass ColumnSpec
3. Validateurs composés
4. Chargeur CSV testable

Synthèse

Une classe sert une abstraction. Un état valide, une interface courte et la composition produisent des pipelines plus testables qu'une hiérarchie complexe.

Source : MIT 6.100L, lectures 17 à 20 ; Fluent Python, chapitres 5, 6, 11 et 13

CHAPITRE 5

Calcul scientifique avec NumPy

Formes, types, mémoire, vectorisation et algèbre

Question directrice

Des transactions contiennent quantités, prix, régions et valeurs manquantes.

Question de compréhension

Quels contrôles précèdent le calcul du chiffre d'affaires ?

Un tableau NumPy

Un `ndarray` possède :

- une forme `shape`
- un nombre d'axes `ndim`
- un type homogène `dtype`
- une organisation mémoire

Inspecter avant de calculer

```
amounts = np.array([12.5, 8.0, 15.5, 4.0])  
print(amounts.shape)    # (4,)  
print(amounts.ndim)     # 1  
print(amounts.dtype)    # float64
```


Vectorisation

```
net = np.array([100.0, 120.0, 80.0])  
gross = net * 1.20  
eligible = gross >= 100  
selected = gross[eligible]
```

Les opérations portent sur un tableau, mais le contrat reste à vérifier.

Boucle Python et boucle compilée

La vectorisation conserve souvent le même nombre d'opérations arithmétiques. Le gain vient de l'exécution compilée, de la représentation compacte et d'un meilleur accès mémoire.

Axes

Pour une matrice observations $\times$ variables :

- `axis=0` réduit les observations et produit une valeur par variable
- `axis=1` réduit les variables et produit une valeur par observation

Question de compréhension

Quelle forme produit `X.mean(axis=0)` si `X.shape == (100, 4)` ?

Broadcasting

```
sales = np.array([[10, 12, 8],  
                  [ 9, 15, 11]]) # (2, 3)  
prices = np.array([4.0, 2.5, 6.0]) # (3,)  
revenue = sales * prices           # (2, 3)
```

L'alignement se fait à partir des dimensions de droite.

Compatible ne signifie pas correct

Deux formes peuvent être compatibles tout en représentant des axes différents. Nommer les axes : semaine, produit, région, variable.

Vue et copie

```
base = np.array([10, 20, 30, 40])
view = base[1:3]
view[0] = 999
print(base)          # [10, 999, 30, 40]

independent = base[1:3].copy()
```

Le grain d'un DataFrame

Écrire une phrase avant le code :

« Une ligne représente une commande terminée unique. »

Le grain détermine la clé, les doublons et le sens des agrégations.

Audit initial

```
print(orders.shape)
print(orders.dtypes)
print(orders.isna().sum())
print(orders["order_id"].nunique())
print(orders["order_id"].duplicated().sum())
```

Comparer chaque résultat à une attente explicite.

Filtrer et affecter

```
valid = orders.loc[orders["unit_price"].notna()].copy()
valid.loc[:, "revenue"] = (
    valid["quantity"] * valid["unit_price"]
)
```

Le `copy()` rend l'indépendance de la table explicite.

Une valeur manquante a un sens

- non mesuré
- non applicable
- non reçu
- réellement nul

Supprimer ou imputer sans distinction change la question.

Taux de manque par groupe

```
missing = (  
    orders.assign(missing_price=orders["unit_price"].isna())  
    .groupby("region", as_index=False)  
    .agg(rows=("order_id", "size"),  
         missing_rate=("missing_price", "mean"))  
)
```

Groupby : trois décisions

1. Quelles clés définissent un groupe ?
2. Quelle mesure résume chaque groupe ?
3. Quel grain possède la table produite ?

Agrégations nommées

```
summary = (  
    valid.groupby("customer", as_index=False)  
        .agg(orders=("order_id", "nunique"),  
             revenue=("revenue", "sum"),  
             mean_basket=("revenue", "mean"))  
)
```

size, count et nunique répondent à des questions différentes.

Risque de jointure

Une clé dupliquée dans la table de référence multiplie les lignes de transactions. Un chiffre d'affaires qui augmente après enrichissement signale souvent une mauvaise cardinalité.

Jointure auditée

```
out = orders.merge(customers, on="customer_id", how="left",  
                   validate="many_to_one", indicator=True)  
assert len(out) == len(orders)  
assert out["_merge"].eq("both").all()
```

Le contrat devient exécutable.

DSA sous les bibliothèques

- index pandas : index algorithmique
- groupby : dictionnaire clé-vers-résumé
- jointure : hachage ou tri
- fenêtre : résumé d'une zone contiguë
- vectorisation : traitement compact d'un bloc numérique

Contrôles d'un pipeline

1. Schéma et grain avant transformation
2. Domaines pendant conversion
3. Cardinalité pendant jointure
4. Comptages et totaux après transformation

Prédire les formes

Avant toute exécution, annoter la forme de chaque objet :

$$X \in \mathbb{R}^{n \times p}, \quad w \in \mathbb{R}^p, \quad Xw \in \mathbb{R}^n.$$

Question de compréhension

Quelle forme de biais produit un score par ligne sans dupliquer les données ?

Déboguer un broadcasting

Une opération qui « fonctionne » peut comparer les mauvais axes.

1. écrire les formes ;
2. aligner les dimensions depuis la droite ;
3. nommer l'axe métier ;
4. tester sur une matrice minuscule calculable à la main.

Réconcilier avant et après

Une transformation de données doit conserver ou expliquer :

- le nombre de lignes ;
- le nombre d'entités distinctes ;
- les totaux métier ;
- la distribution des catégories et des valeurs manquantes.

Travail appliqué

1. Score matriciel vectorisé
2. Audit générique d'une table
3. Ventes par région et mois
4. Jointure avec réconciliation

Synthèse

La préparation fiable repose sur les formes, les types, le grain, les clés et les cardinalités.
Une ligne de code concise ne remplace aucun de ces contrôles.

Source : Documentation officielle NumPy et pandas ; Fluent Python

Anatomie d'un ndarray

- tampon typé;
- shape et ndim;
- dtype et itemsize;
- strides;
- vues partageant la mémoire.

Estimation : `size * itemsize` octets de données.

Construire avec intention

```
grid = np.linspace(0., 1., num=101)
X = np.zeros((10_000, 8), dtype=np.float32)
I = np.eye(4)
```

empty alloue sans initialiser ; linspace fixe le nombre de points.

Le dtype est une hypothèse

- domaine des valeurs ;
- précision ;
- mémoire ;
- comportement du débordement ;
- compatibilité des opérations.

Pour les flottants, tester une tolérance avec `isclose`.

Reshape et axes

$X \in \mathbb{R}^{n \times p}$ n'est pas seulement « un tableau 2D ».

- axe 0 : observations ;
- axe 1 : variables ;
- reshape conserve le nombre d'éléments ;
- transpose permute le sens des axes.

Réductions avec keepdims

Pour centrer chaque variable :

$$\mu = X.\text{mean}(\text{axis} = 0, \text{keepdims} = \text{True}), \quad X_c = X - \mu.$$

La forme $(1, p)$ documente le broadcasting.

Softmax numériquement stable

```
row_max = X.max(axis=1, keepdims=True)
exp = np.exp(X - row_max)
prob = exp / exp.sum(axis=1, keepdims=True)
```

Même formule mathématique, meilleur comportement numérique.

Résoudre, ne pas inverser

Pour $Ax = b$: `np.linalg.solve(A, b)`.

- plus direct ;
- généralement plus stable ;
- valider par le résidu $\|Ax - b\|$.

Moindres carrés : `np.linalg.lstsq`.

Un générateur explicite

```
rng = np.random.default_rng(2026)
samples = rng.normal(loc=0, scale=1,
                     size=100_000)
```

Passer rng aux fonctions : dépendance visible et tests reproductibles.

Vectorisation et explosion mémoire

Distances (n, p) contre centroïdes (m, p) :

- broadcasting direct : intermédiaire (n, m, p) ;
- identité algébrique : résultat (n, m) ;
- blocs : mémoire bornée.

La ligne la plus courte n'est pas toujours la meilleure.

Checklist NumPy

Nommer les axes, prédire la forme, inspecter le dtype, distinguer vue et copie, estimer les temporaires, stabiliser la formule, vérifier puis mesurer.

CHAPITRE 6

Ingénierie tabulaire et EDA avec pandas

Schéma, index, transformations, temps et preuves descriptives

Question directrice

Les livraisons semblent plus lentes.

Question de compréhension

Quelles explications concurrentes faut-il distinguer avant de calculer une moyenne ?

Population et grain

Une ligne peut représenter :

- une commande
- une tentative de livraison
- un article
- un client

Le même calcul reçoit un sens différent pour chaque grain.

Audit avant résumé

- période et population
- nombre de lignes et clés uniques
- doublons et valeurs impossibles
- types, unités et valeurs manquantes

Moyenne et médiane

$$\bar{x} = \frac{1}{n} \sum_i x_i$$

La moyenne utilise toutes les valeurs. La médiane résiste davantage à quelques extrêmes. Le choix dépend de la question, pas d'une règle universelle.

Une observation influente

[20, 21, 22, 23, 120]

- moyenne : 41,2 minutes

- médiane : 22 minutes

Question de compréhension

Quelle mesure décrit une livraison typique ? Laquelle décrit la charge totale moyenne ?

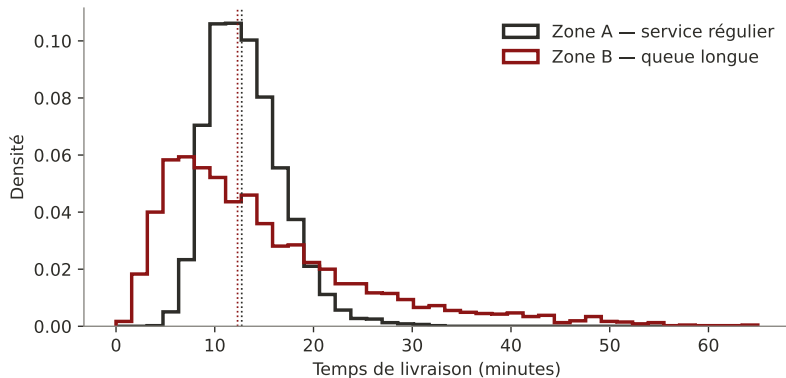
Dispersion

- écart type : unité d'origine, sensible aux extrêmes
- intervalle interquartile : largeur de la moitié centrale
- quantiles : seuils opérationnels, par exemple le 90e percentile

Résumé adapté

```
summary = durations.describe(  
    percentiles=[.05, .25, .5, .75, .9, .95]  
)  
iqr = durations.quantile(.75) - durations.quantile(.25)
```


Distribution des durées



Valeur atypique ou incident réel ?

La règle IQR signale des observations à examiner. Elle ne démontre ni erreur, ni fraude, ni absence de pertinence.

Valeurs manquantes

Le taux global peut masquer une concentration par région, canal ou semaine.
Transformer l'absence en indicatrice permet d'analyser le processus de collecte.

Manque par région

```
profile = (  
    df.assign(missing_distance=df["distance_km"].isna())  
    .groupby("region", as_index=False)  
    .agg(rows=("delivery_id", "size"),  
         missing=("missing_distance", "mean"))  
)
```

Comparer des groupes

Pour chaque groupe, présenter :

- effectif
- centre pertinent
- dispersion ou quantiles
- taux de données manquantes

Une moyenne seule cache la qualité et la précision.

Résumé par région

```
by_region = (  
    df.groupby("region", as_index=False)  
        .agg(n=("delivery_id", "nunique"),  
             median=("duration_min", "median"),  
             q90=("duration_min", lambda s: s.quantile(.9)))  
)
```

Covariance et corrélation

La corrélation de Pearson décrit une association linéaire normalisée. Elle réagit aux extrêmes et ne démontre pas une causalité.

Association conditionnelle

Une région peut avoir à la fois des distances plus longues et des durées plus longues. Stratifier par distance vérifie si l'association région-durée subsiste à composition plus comparable.

Nuage de points informatif

```
sns.scatterplot(data=df,  
                x="distance_km", y="duration_min",  
                hue="region", alpha=.55)  
plt.xlabel("Distance (km)")  
plt.ylabel("Duree (minutes)")
```

Axes, unités et groupes rendent la question visible.

Choisir un graphique

Question	Graphique
distribution	histogramme ou ECDF
comparaison de groupes	boîte ou violon
relation numérique	nuage de points
évolution	série temporelle

Analyse de sensibilité

Comparer des choix raisonnables :

- moyenne et médiane
- avec et sans erreurs confirmées
- période complète et récente
- agrégé et stratifié

Une conclusion instable doit être signalée.

Enquête sur les retards

1. Valider le grain
2. Comparer volumes et manque dans le temps
3. Examiner distribution globale et groupes
4. Étudier quantiles avant et après changement
5. Qualifier les extrêmes
6. Limiter la conclusion aux preuves disponibles

Lire le graphique comme un argument

Un graphique utile permet d'énoncer :

1. ce qui est comparé ;
2. l'écart visible ;
3. l'incertitude ou la variabilité ;
4. ce que la figure ne permet pas de conclure.

Quand le résumé change l'histoire

La moyenne répond à une question additive ; la médiane décrit une observation centrale. Pour une distribution asymétrique, elles peuvent soutenir des décisions différentes.

Question de compréhension

Laquelle utiliseriez-vous pour une promesse de délai client ?

Transformer une anomalie en enquête

Une valeur extrême peut être :

- une erreur de mesure ;
- un cas rare mais réel ;
- un changement de population ;
- un incident opérationnel à forte valeur métier.

La supprimer sans diagnostic détruit potentiellement le signal recherché.

Travail appliqué 1/2

1. Profil complet des livraisons
2. Influence des observations IQR

Chaque réponse contient contrôle, tableau, graphique et interprétation.

Travail appliqué 2/2

- 3. Structure des valeurs manquantes
- 4. Corrélation globale et par groupe

Synthèse

L'EDA relie une question, un grain, des contrôles, des résumés et des graphiques. Elle découvre des hypothèses ; elle ne prouve pas automatiquement une cause.

Source : Documentation pandas, seaborn et SciPy

Series, DataFrame, Index

- valeurs ;
- étiquettes ;
- alignement ;
- types par colonne.

Question avant toute opération : relation par position, index ou clé métier ?

Schéma à la lecture

- `usecols` ;
- `dtype` ;
- `parse_dates` ;
- valeurs manquantes ;
- conversions échouées.

Lire un fichier est déjà une décision de modélisation.

Types pandas

`string`, `boolean`, `Int64`, `category`, `datetime`, `period`.

Pourquoi ?

Domaine plus clair, absence représentable, mémoire et opérations adaptées.

Affectation et Copy-on-Write

Éviter l'affectation chaînée.

- une seule opération `loc` ;
- ou un nouvel objet par `assign` ;
- ne pas compter sur la mutation d'un objet intermédiaire.

Source : Guide pandas, Copy-on-Write

Long, large et grain

`melt` colonnes vers lignes ;

`pivot` une valeur par cellule exigée ;

`pivot_table` agrégation explicitement choisie.

Un pivot qui échoue révèle souvent un grain mal compris.

Trois contrats GroupBy

- `agg` : grain réduit ;
- `transform` : une valeur alignée par ligne ;
- `filter` : groupes entiers conservés ;
- `apply` : général, contrat moins visible.

Mathématique d'une jointure

Pour une clé k , a_k lignes à gauche et b_k à droite donnent $a_k b_k$ lignes dans une jointure interne.

Contrôles

validate, indicator, lignes, clés, somme métier.

Le temps n'est pas une chaîne

- instant UTC ;
- fuseau local ;
- date civile ;
- période ;
- durée.

Les changements d'heure et jours partiels influencent les agrégats.

Fenêtres temporelles

Sept observations $\neq$ sept jours.

- `resample("D");`
- calendrier complet ;
- politique zéro ou manquant ;
- `rolling("7D");`
- frontière et données tardives.

Chaînes testables

pipe nomme les étapes ; assign construit de nouvelles colonnes ; loc filtre.

Limite

Découper une chaîne lorsque les contrats et audits deviennent invisibles.

Passage à l'échelle

Sélectionner les colonnes, filtrer tôt, choisir les dtypes, éviter `iterrows`, mesurer `apply`, traiter par chunks et fusionner un état associatif.

Checklist pandas

Grain, schéma, index, absence, unicité, cardinalité, transformation, temps, réconciliation et table d'audit.

Source : Documentation officielle pandas : GroupBy, merge, Copy-on-Write

CHAPITRE 7

SciPy, probabilités et simulation

Distributions, intégration, optimisation et modèles de décision

Question directrice

Combien de produits commander avant de connaître la demande ?

Question de compréhension

Quelles informations manquent à la seule demande moyenne ?

Variable aléatoire

Une variable aléatoire associe un nombre à chaque résultat possible.

- distribution : valeurs possibles et probabilités
- espérance : moyenne théorique
- variance : dispersion des scénarios

Bernoulli

$X = 1$ si un achat a lieu et $X = 0$ sinon.

$$P(X = 1) = p, \quad E[X] = p, \quad \text{Var}(X) = p(1 - p)$$

La moyenne d'indicateurs est une proportion.

Générateur reproductible

```
rng = np.random.default_rng(2026)
visits = rng.binomial(n=1, p=.08, size=10_000)
estimated_rate = visits.mean()
```

La graine reproduit le calcul ; elle ne supprime pas l'incertitude Monte-Carlo.

Probabilité conditionnelle

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Le dénominateur devient l'ensemble des situations où B est observé.

Indépendance

A et B sont indépendants si connaître B ne change pas la probabilité de A .

$$P(A \mid B) = P(A)$$

Indépendance est une hypothèse du modèle, pas une propriété obtenue par syntaxe.

Bayes et taux de base

Prévalence 1%, sensibilité 95%, faux positifs 5%.

- 95 vrais positifs sur 10 000 dossiers
- 495 faux positifs attendus
- environ 16% des alertes sont de vrais cas

Calcul de Bayes

```
posterior = (  
    sensitivity * prevalence  
    / (sensitivity * prevalence  
        + false_positive_rate * (1 - prevalence))  
)
```

Question de compréhension

Que devient le résultat si la prévalence passe à 10% ?

Choisir une distribution

Loi	Mécanisme
Bernoulli	un événement oui/non
Binomiale	succès sur n essais
Poisson	nombre d'événements par exposition
Exponentielle	attente entre événements
Normale	variation continue agrégée

Le mécanisme avant la forme

Un histogramme ressemblant à une cloche ne suffit pas pour imposer une normale. Les unités, le support, la dépendance et la stabilité du taux font partie du modèle.

Structure Monte-Carlo

1. Définir les hypothèses
2. Générer un scénario
3. Calculer la quantité d'intérêt
4. Répéter
5. Résumer et tester la sensibilité

Risque de rupture

```
demand = rng.poisson(lam=42, size=100_000)
stock = 50
p_shortage = np.mean(demand > stock)
waste = np.mean(np.maximum(stock - demand, 0))
shortage = np.mean(np.maximum(demand - stock, 0))
```

Une décision exige des coûts

Deux stocks peuvent avoir la même probabilité de rupture et des coûts très différents.
Comparer invendus, ruptures et coût attendu.

Choix du stock

```
costs = {}  
for stock in range(35, 61):  
    waste = np.maximum(stock - demand, 0)  
    shortage = np.maximum(demand - stock, 0)  
    costs[stock] = np.mean(waste + 4 * shortage)  
best = min(costs, key=costs.get)
```

Temps et mémoire de simulation

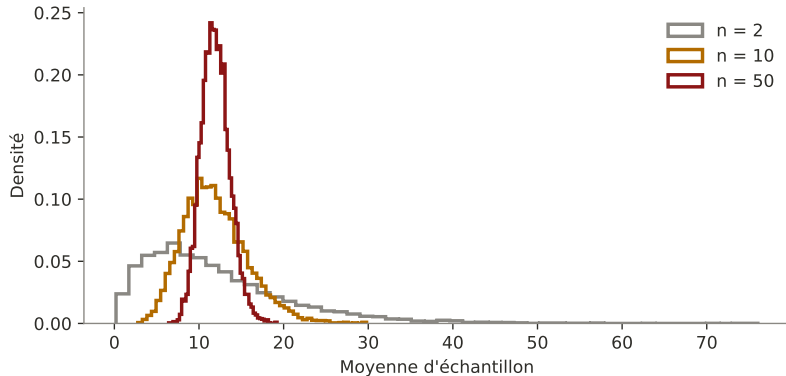
Pour B scénarios et k décisions :

- temps $O(Bk)$
- mémoire $O(B + k)$ avec un vecteur de scénarios
- mémoire $O(Bk)$ si toute la matrice est conservée

Loi des grands nombres

La moyenne simulée se stabilise lorsque le nombre de répétitions augmente. Elle se stabilise autour de l'espérance du modèle choisi, pas forcément autour de la réalité si le modèle est mauvais.

Moyennes d'échantillons



Théorème central limite

Sous des conditions appropriées, la distribution de nombreuses moyennes devient approximativement normale et son écart type diminue comme $1/\sqrt{n}$. La dépendance et les queues lourdes demandent de la prudence.

Concevoir une simulation crédible

1. définir l'unité aléatoire ;
2. écrire le mécanisme générateur ;
3. définir la quantité observée ;
4. répéter avec une graine contrôlée ;
5. quantifier l'erreur Monte-Carlo.

Erreur Monte-Carlo

L'incertitude numérique décroît typiquement comme $1/\sqrt{B}$. Multiplier le nombre de répétitions par 100 divise donc l'erreur par environ 10, pas par 100.

Taux de base et test médical

Même un test très sensible peut produire beaucoup de faux positifs lorsque la maladie est rare.

Question de compréhension

Faut-il raisonner depuis $P(+ | M)$ ou depuis $P(M | +)$?

Sensibilité du modèle de stock

Faire varier séparément :

- la distribution de la demande ;
- le coût d'une rupture ;
- le coût d'un invendu ;
- la dépendance entre jours.

Une décision stable sous ces variations est plus défendable.

Quand ne pas croire la simulation

Davantage de répétitions ne corrige ni une probabilité mal estimée, ni une dépendance ignorée, ni une décision mal formulée.

Question de compréhension

Quelle hypothèse mérite d'abord une collecte de données ?

Travail appliqué

1. Somme de deux dés
2. Stock du café
3. Test médical et taux de base
4. Moyennes d'échantillons exponentiels

Synthèse

La simulation répond à une question sous un modèle. Les hypothèses décrivent le monde généré ; l'analyse de sensibilité montre lesquelles commandent la décision.

Source : Documentation NumPy et SciPy ; MIT 6.100L, simulation

SciPy par familles de problèmes

- stats : distributions et tests ;
- optimize : minima et racines ;
- integrate : quadrature et dynamique ;
- linalg, sparse, interpolate, signal.

Nommer le problème avant la fonction.

Objet distribution

Une loi figée expose : pdf/pmf, cdf, sf, ppf, rvs, moments.

Queue

Préférer $\text{sf}(x)$ à $1 - \text{cdf}(x)$ lorsque la précision numérique compte.

Gamma appliquée à un délai

```
model = stats.gamma(a=2.5, scale=8.0)
samples = model.rvs(size=100_000,
                    random_state=rng)
p_over_30 = model.sf(30)
q95 = model.ppf(.95)
```

Lire les conventions de paramètres et leurs unités.

Ajuster n'est pas valider

- calibrer sur un sous-échantillon ;
- valider ailleurs ;
- comparer ECDF et quantiles ;
- inspecter la queue utile ;
- chercher mélanges, dépendance et censure.

Test statistique = contrat

Unité indépendante + estimande + hypothèse nulle + statistique + distribution de référence + conditions.

Exemple

Welch compare deux moyennes indépendantes sans imposer l'égalité des variances.

Permutation

Sous l'hypothèse nulle, les étiquettes doivent être échangeables.

1. mélanger ;
2. recouper ;
3. recalculer ;
4. compter les statistiques aussi extrêmes.

L'ordinateur n'invente pas l'échangeabilité.

Intégration ou Monte-Carlo ?

	Quadrature	Monte-Carlo
Atout	faible dimension, régularité	simulateur complexe, haute dimension
Erreur	approximation numérique	erreur Monte-Carlo
Validation	tolérance et cas connu	convergence et répliques

Optimiser sous contraintes

```
result = optimize.minimize_scalar(  
    expected_cost,  
    bounds=(0, 100), method="bounded")  
if not result.success:  
    raise RuntimeError(result.message)
```

Vérifier le voisinage et convertir prudemment vers une décision entière.

Un optimiseur ne comprend pas le métier

Il minimise exactement l'objectif codé.

- domaine ;
- unités ;
- contraintes ;
- bruit de simulation ;
- optimum local ;
- message de convergence.

Matrices creuses

Dense : espace $O(np)$. Creux : espace lié aux z non-zéros et à leurs indices.

Piège

Centrer une matrice creuse peut la densifier.

Trois incertitudes distinctes

1. erreur numérique de l'algorithme ;
2. erreur Monte-Carlo ;
3. incertitude du modèle et de ses paramètres.

Augmenter le nombre de simulations ne traite que la deuxième.

Checklist SciPy

Problème mathématique, hypothèses, unités, domaine, routine, convergence, cas connu, méthode indépendante, sensibilité.

Source : Guide officiel SciPy ; documentation `scipy.stats`

CHAPITRE 8

Inférence fréquentiste et bayésienne

statsmodels, PyMC, diagnostics et décision A/B

Question directrice

La conversion passe de 8,0% à 8,4%.

Question de compréhension

Quelles informations manquent avant de décider un déploiement ?

Estimande

La quantité exacte visée doit être nommée :

- différence absolue de proportions
- différence de moyennes
- effet médian
- ratio de deux taux

La méthode dépend de cette définition.

Unité indépendante

Si l'expérience randomise les utilisateurs, l'analyse doit respecter les utilisateurs. Traiter chaque clic comme indépendant sous-estime l'incertitude des utilisateurs actifs.

Erreur standard d'une proportion

$$SE(\hat{p}) \approx \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$$

À taux similaire, multiplier n par 100 divise environ l'erreur par 10.

Intervalle de confiance

Un intervalle décrit la variabilité de la procédure d'estimation sous des répétitions du mécanisme d'échantillonnage. Il ne donne pas directement la probabilité que ce paramètre fixe soit dans cet intervalle.

Bootstrap

1. Tirer les unités avec remise
2. Recalculer la statistique
3. Répéter B fois
4. Prendre des quantiles des estimations

Bootstrap de différence

```
for b in range(repeats):  
    c = rng.choice(control, len(control), replace=True)  
    t = rng.choice(treatment, len(treatment), replace=True)  
    estimates[b] = t.mean() - c.mean()  
interval = np.quantile(estimates, [.025, .975])
```

Coût du bootstrap

- boucle par répétition : deux échantillons temporaires
- vecteur final : $O(B)$
- matrice de tous les tirages : $O(Bn)$

Rééchantillonner l'unité indépendante manipulée par le design.

Hypothèse nulle

Le test définit une statistique et sa distribution sous H_0 . La question : les données seraient-elles surprenantes si le modèle nul était vrai ?

Valeur p

La valeur p n'est pas :

- la probabilité que H_0 soit vraie
- la taille de l'effet
- la probabilité que l'expérience se reproduise
- une décision métier

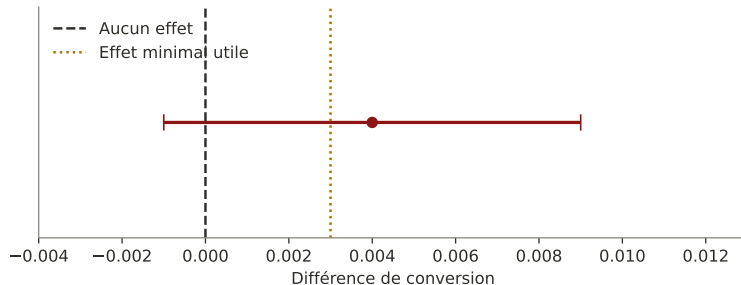
Quatre lectures

1. Estimation de l'effet
2. Incertitude de l'estimation
3. Compatibilité avec le modèle nul
4. Valeur pratique pour la décision

Seuil d'effet utile

Définir avant l'analyse le gain minimal qui justifie le coût et le risque du changement.
Un effet statistiquement détectable peut rester trop petit pour être utile.

Estimation et seuil utile



Avant le lancement

- unité de randomisation et population
- métrique principale et gardes
- effet minimal utile
- durée et règle d'arrêt
- exclusions et contrôles

Intégrité A/B

- ratio d'allocation
- utilisateur dans un seul groupe
- événements uniques
- exposition réellement reçue
- données manquantes et contamination

Résumé au bon grain

```
summary = (  
    experiment.groupby("variant", as_index=False)  
        .agg(users=("user_id", "nunique"),  
             conversions=("converted", "sum"))  
)  
summary["rate"] = summary["conversions"] / summary["users"]
```

Note de décision

Une conclusion contient :

- effet et intervalle
- comparaison à zéro et au seuil utile
- résultat des contrôles d'intégrité
- métriques de garde
- prochaine action justifiée

Puissance

La puissance augmente avec la taille d'échantillon, la taille d'effet et un bruit plus faible.
La taille doit être planifiée avant de regarder le résultat.

Multiplicité

Tester de nombreux segments et s'arrêter au premier $p < 0.05$ augmente les faux signaux. Nommer les analyses exploratoires et les confirmer sur de nouvelles données.

Interpréter un intervalle

Un intervalle de confiance quantifie l'incertitude de la procédure d'estimation. Il ne donne pas la probabilité que ce paramètre fixe appartienne à cet intervalle particulier dans l'interprétation fréquentiste.

Interpréter une p-value

La p-value est calculée sous l'hypothèse nulle. Elle n'est ni $P(H_0 \mid \text{données})$, ni la taille de l'effet, ni la probabilité que le résultat se reproduise.

L'unité d'analyse doit suivre le design

Si l'affectation se fait par utilisateur mais les lignes sont des clics, traiter les clics comme indépendants sous-estime l'incertitude.

Question de compréhension

À quel niveau faut-il agréger ou ré-échantillonner ?

Décider avec un seuil utile

Un effet statistiquement compatible avec une amélioration peut rester trop petit pour financer le déploiement. Comparer l'intervalle à un seuil de valeur métier, pas seulement à zéro.

Travail appliqué

1. Intervalle d'une proportion
2. Bootstrap au niveau utilisateur
3. Audit d'un jeu A/B imparfait
4. Note de décision complète

Synthèse

L'inférence relie estimation et incertitude. L'expérience crédible dépend de son design et de son intégrité ; la décision ajoute valeur pratique, gardes et coût d'information.

Source : Documentation SciPy et statsmodels

Le modèle bayésien génératif

$$p(\theta \mid y) \propto p(y \mid \theta)p(\theta)$$

- prior : mondes plausibles avant données ;
- vraisemblance : mécanisme d'observation ;
- postérieure : incertitude conditionnelle au modèle.

Workflow PyMC

Formuler $\rightarrow$ prior prédictif $\rightarrow$ échantillonner $\rightarrow$ diagnostiquer $\rightarrow$ posterior prédictif $\rightarrow$ sensibilité $\rightarrow$ décision.

Ordre essentiel

On ne résume pas la postérieure avant d'avoir diagnostiqué le calcul.

Modèle bêta–binomial

```
with pm.Model(coords={"variant": ["A", "B"]}):  
    rate = pm.Beta("rate", 2, 18, dims="variant")  
    y = pm.Binomial("y", n=trials, p=rate,  
                    observed=successes, dims="variant")  
    prior = pm.sample_prior_predictive(random_seed=2026)  
    idata = pm.sample(2000, tune=2000, chains=4,  
                      target_accept=.9, random_seed=2026)
```

Prior interprétable

Pour $\text{Beta}(\alpha, \beta)$: moyenne $\alpha/(\alpha + \beta)$ et concentration $\alpha + \beta$.

Question de compréhension

Quelles conversions le prior prédictif rend-il plausibles ?

Prior prédictif

Tirer paramètres, puis observations avant de regarder les données.

- plage ;
- moyenne ;
- dispersion ;
- queues ;
- cas impossibles.

Un prior « vague » sur un coefficient peut être extrême sur la sortie.

NUTS et chaînes

- tune adapte l'échantillonneur ;
- plusieurs chaînes explorent depuis différents départs ;
- autocorrélation réduit l'information effective ;
- plus de tirages ne répare pas un mauvais modèle.

Diagnostics avant résultat

R-hat cohérence entre chaînes ;

ESS information effective ;

divergences géométrie mal explorée ;

traces mélange, dérive, modes ;

énergie efficacité dynamique.

Audit ArviZ

```
summary = az.summary(idata, var_names=["rate"],  
                      hdi_prob=.95)  
az.plot_trace(idata, var_names=["rate"])  
divergences = int(  
    idata.sample_stats["diverging"].sum())
```

Une divergence n'est pas une décoration à ignorer.

Posterior predictive check

Le modèle ajusté reproduit-il :

- moyenne et dispersion ;
- zéros et extrêmes ;
- différences de groupes ;
- structure temporelle ?

Un bon coefficient ne garantit pas de bonnes données simulées.

Quantités de décision

```
post = idata.posterior["rate"]
delta = (post.sel(variant="B")
         - post.sel(variant="A"))
p_positive = float((delta > 0).mean())
p_useful = float((delta > .003).mean())
```

Positif et utile sont deux événements différents.

Solution analytique comme test

Pour le modèle bêta–binomial :

$$p \mid y \sim \text{Beta}(\alpha + y, \beta + n - y).$$

Comparer PyMC à la solution fermée valide le workflow sur un cas connu.

Trois stratégies régionales

Séparé variance forte dans petits groupes ;

Complet même taux imposé ;

Hiérarchique pooling partiel appris.

Le shrinkage est plus fort lorsque l'information locale est faible.

Paramétrisation non centrée

$$\eta_r = \mu + \sigma z_r, \quad z_r \sim \mathcal{N}(0, 1).$$

Souvent plus facile à échantillonner pour groupes peu informatifs. L'hypothèse d'échangeabilité doit rester défendue.

Coefficient ou effet métier ?

Un coefficient log-odds n'est pas un effet en points de probabilité.

Standardisation

Prédire chaque unité sous traitement et contrôle, puis moyenner la différence.

Sensibilité

Varier priors, vraisemblance, règles d'inclusion et unités. Une décision instable identifie l'hypothèse qui mérite information ou prudence.

Checklist PyMC

Modèle génératif, prior prédictif, grain, chaînes, R-hat, ESS, divergences, traces, posterior prédictif, décision, sensibilité, versions.

Source : Documentation officielle PyMC et ArviZ

CHAPITRE 9

Systèmes de machine learning avec scikit-learn

Pipelines, sélection, interprétation et exploitation

Question directrice

Une équipe peut appeler 500 clients par semaine pour prévenir l'attrition.

Question de compréhension

Que doit produire le modèle pour soutenir cette décision ?

Formulation temporelle

- unité : client actif à la date t
- cible : résiliation dans les 60 jours
- variables : disponibles avant ou à t
- décision : sélectionner les 500 scores les plus élevés

Fuite de cible

Un statut de clôture, un remboursement futur ou un agrégat utilisant toute la période révèle des informations postérieures à la prédiction. La date d'extraction ne prouve pas la disponibilité opérationnelle.

Séparation des données

- aléatoire pour des observations échangeables
- temporelle pour simuler le futur
- par groupe pour éviter qu'un même utilisateur traverse les partitions

Train et test

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=.25,  
    stratify=y, random_state=2026  
)
```

Le test reste fermé pendant le choix des variables, du modèle et du seuil.

Baseline

Une baseline répond à : « que savons-nous faire sans le nouveau modèle ? »

- classe majoritaire
- taux historique
- règle métier simple

Accuracy et classe rare

Si 2% des clients résilient, prédire « non » pour tous donne 98% d'accuracy et ne trouve aucun client à contacter.

Question de compréhension

Quelle métrique révèle immédiatement cet échec ?

Prétraitement appris

L'imputation, la normalisation et les catégories observées possèdent un état appris. Cet état doit être ajusté uniquement sur les données d'entraînement.

Pipeline scikit-learn

```
model = Pipeline([
    ("preprocess", preprocess),
    ("classifier", LogisticRegression(max_iter=1000)),
])
model.fit(X_train, y_train)
```

Le même objet applique les mêmes transformations en validation, test et production.

Régression ou classification

Tâche	Sortie	Exemple
régression	quantité numérique	coût d'un sinistre
classification	catégorie ou score	risque d'attrition

Métriques de régression

- MAE : erreur absolue moyenne dans l'unité de la cible
- RMSE : poids plus fort pour les grandes erreurs
- R^2 : comparaison au prédicteur moyen

Matrice de confusion

	prédit négatif	prédit positif
réel négatif	VN	FP
réel positif	FN	VP

Précision et rappel

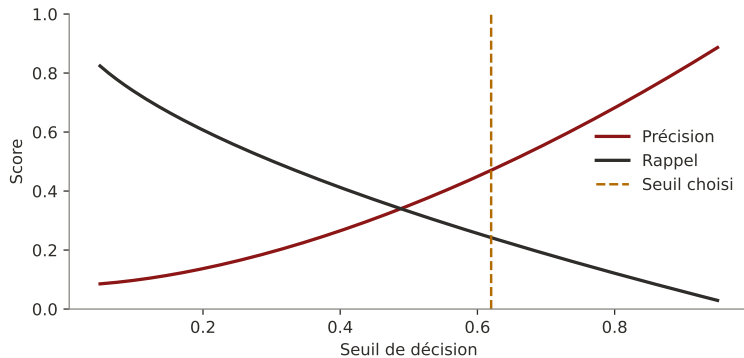
$$\text{précision} = \frac{VP}{VP + FP}, \quad \text{rappel} = \frac{VP}{VP + FN}$$

Précision : qualité des alertes. Rappel : couverture des cas réels.

Le seuil crée la décision

Un score de 0,42 n'est ni une classe ni une action. Le seuil dépend du coût des erreurs, de la capacité et du bénéfice attendu.

Compromis précision-rappel



Capacité de 500 dossiers

```
capacity = 500
selected = np.argsort(score)[-capacity:]
precision_at_capacity = y_test.iloc[selected].mean()
covered = y_test.iloc[selected].sum() / y_test.sum()
```

Le seuil est le score du dossier à la frontière.

ROC-AUC et Average Precision

- ROC-AUC : qualité globale du classement
- Average Precision : plus informative pour une classe rare
- métrique à capacité : proche de l'action réelle

Une seule métrique ne répond pas à toutes les décisions.

Surapprentissage

Un grand écart entre entraînement et validation signale que le modèle retient des particularités qui se généralisent mal. Comparer les distributions des scores entre partitions.

Validation croisée

La validation croisée répète l'entraînement sur plusieurs partitions internes. Elle aide au choix du modèle et montre la variabilité ; elle ne remplace pas le test final.

Analyse des erreurs

Examiner faux positifs et faux négatifs avec :

- score et cible
- segment et période
- variables disponibles
- qualité des données

Interprétation

Les coefficients et importances décrivent le comportement du modèle. Ils ne démontrent pas un mécanisme causal et dépendent des variables corrélées et du jeu sur lequel ils sont calculés.

Calibration et classement

Un modèle peut bien classer les dossiers sans fournir des probabilités fiables.

- discrimination : ordonner les risques ;
- calibration : faire correspondre score et fréquence observée ;
- décision : convertir le score selon coûts et capacité.

Livvable final

1. Formulation et dictionnaire de données
2. Audit, baseline et séparation
3. Pipeline et modèle
4. Métriques globales et opérationnelles
5. Analyse des erreurs et limites
6. Recommandation de déploiement ou d'expérimentation

Travail appliqué

1. Détecter la fuite avec une chronologie
2. Comparer trois baselines
3. Construire un pipeline sans fuite
4. Choisir un seuil sous capacité

Synthèse

L'évaluation crédible simule l'usage futur. La formulation empêche la fuite, la baseline donne un repère, le pipeline protège l'état appris et le seuil relie score et action.

Source : Documentation scikit-learn ; Müller et Guido

Contrat d'un estimateur

`fit` apprend l'état ;

`transform` change la représentation ;

`predict` produit une décision ;

`predict_proba` produit des scores ;

`get_params` expose la configuration.

Ce contrat rend les objets clonables en validation.

Le split modélise le futur

- i.i.d. : aléatoire/stratifié ;
- plusieurs lignes par entité : groupé ;
- futur après passé : temporel ;
- nouveaux sites : validation par site.

Le test reste hors de toute boucle de choix.

ColumnTransformer complet

```
preprocess = ColumnTransformer([
    ("num", numeric_pipeline, numeric_columns),
    ("cat", categorical_pipeline, categorical_columns),
])
model = Pipeline([
    ("preprocess", preprocess),
    ("classifier", LogisticRegression(max_iter=2000)),
])
```

Catégorie inconnue

`handle_unknown="ignore"` évite l'échec, mais transforme la nouvelle catégorie en absence d'indicateur.

Monitoring

Mesurer le taux de catégories inconnues par variable et période.

Validation croisée : une distribution

Conserver par fold : score train, score validation, durée, taille, prévalence et groupes.

Question de compréhension

Que signifie un seul fold catastrophique ?

Hyperparamètres sous budget

- recherche aléatoire pour plusieurs dimensions ;
- prétraitement dans le pipeline ;
- split respectant groupes/temps ;
- score choisi avant ;
- test final intact.

Validation imbriquée

Boucle interne : choisir configuration. Boucle externe : estimer la procédure de choix.

Pourquoi ?

De nombreux essais rendent le meilleur score de validation optimiste.

Attention aux scorers négatifs

scikit-learn maximise tous les scorers.

$$\text{neg_mean_absolute_error} = -3 \iff \text{MAE} = 3.$$

Rapporter dans l'unité métier.

Classes rares

- poids de classe modifie la perte ;
- sur-échantillonnage seulement dans le fold train ;
- accuracy souvent trompeuse ;
- calibration peut changer ;
- mesurer volume au seuil.

Calibration

Parmi les scores proches de 0,2, environ 20% devraient être positifs.

- courbe de calibration ;
- score de Brier ;
- calibration sur données séparées ;
- suivi selon période et segment.

Interpréter sans causalité

- coefficient : dépend de l'échelle ;
- permutation : dépend du score et du jeu ;
- corrélation entre variables partage l'importance ;
- PDP/ICE peuvent explorer des zones irréalistes.

Incertitude d'une métrique

Rééchantillonner l'unité indépendante. Pour comparer deux modèles, bootstrapper la différence sur les mêmes unités.

Limite

Un intervalle ne répare pas un split qui fuit.

Sérialisation = contrat de versions

Conserver : code, artefact fiable, versions, schéma, ordre des colonnes, catégories, seuil, empreinte des données et lot de compatibilité.

Monitoring à quatre étages

1. qualité des entrées ;
2. distributions et scores ;
3. latence, erreurs, capacité ;
4. performance retardée et valeur.

Chaque alerte a un propriétaire et une action.

Boucle de rétroaction

L'action du modèle change les labels futurs. Contacter un client peut empêcher l'attrition.

Conséquence

Prévoir contrôle expérimental ou méthode causale ; ne pas réentraîner aveuglément.

Checklist système ML

Temps, split, baseline, pipeline, validation, seuil, incertitude, segments, artefact, contrat d'entrée, monitoring et retrait.

Source : Guide scikit-learn : pipelines et estimateurs composites

CHAPITRE 10

Bonus — Applications GenAI avec Hugging Face

Modèles, données, évaluation et garde-fous

Question directrice

Transformer un incident libre en :

- catégorie ;
- résumé fidèle ;
- prochaine action proposée.

Le système doit aussi expliquer sa latence, son coût, ses erreurs et sa frontière de données.

Un modèle n'est pas une application

1. Données et politique
2. Tokenizer
3. Modèle et inférence
4. Validation
5. Décision
6. Mesure

Chaque étage possède ses propres modes d'échec.

Écosystème Hugging Face

Hub dépôts, révisions, cartes, poids et tokenizers ;
Transformers chargement et inférence locale ;
Datasets chargement, transformations, streaming ;
huggingface_hub téléchargement et client d'inférence.

Source : Documentation officielle Hugging Face

Lire une model card

Licence	autorise-t-elle l'usage prévu ?
Langues	le français et le domaine sont-ils évalués ?
Taille	mémoire, latence et matériel compatibles ?
Contexte	entrée + preuves + sortie tiennent-elles ?
Limites	quels biais et cas d'échec sont documentés ?
Révision	quelle version exacte sera déployée ?

Pipeline local

```
from transformers import pipeline

generator = pipeline(
    "text-generation",
    model="HuggingFaceTB/SmolLM2-1.7B-Instruct",
    device_map="auto",
)
result = generator(messages, max_new_tokens=80,
                    do_sample=False)
```

Contrôle des poids et des données ; coût matériel et temps de chargement.

Inférence hébergée

```
import os
from huggingface_hub import InferenceClient

client = InferenceClient(provider="auto",
    api_key=os.environ["HF_TOKEN"], timeout=30)
response = client.chat.completions.create(
    model="your-approved-model-id",
    messages=messages, max_tokens=120, temperature=0)
```

Source : Hugging Face, InferenceClient

Local ou hébergé ?

Comparer sur le même corpus :

- qualité et versions ;
- latence, débit et disponibilité ;
- mémoire et coût par requête ;
- confidentialité et rétention ;
- charge opérationnelle.

Le contrat du produit décide.

Tokenisation

- Le texte devient une suite d'identifiants.
- Mots et tokens ne coïncident pas.
- Le contexte contient prompt, historique, preuves et sortie.
- Une troncature silencieuse change le problème.

Inspecter le tokenizer

```
tokenizer = AutoTokenizer.from_pretrained(model_id)
encoded = tokenizer(text, return_tensors="pt",
                    truncation=True, max_length=128)
print(encoded["input_ids"].shape)
print(tokenizer.convert_ids_to_tokens(
    encoded["input_ids"][0]))
```

Mesurer les tokens ; ne pas déduire le budget des caractères.

Chat template

Le modèle attend le format appris : rôles, séparateurs et marqueur de génération.

Erreur classique

Concaténer manuellement User: ... Assistant: sans vérifier le template du tokenizer.

Paramètres de génération

`max_new_tokens` borne la sortie ;

`do_sample` active l'échantillonnage ;

`temperature` modifie la dispersion ;

`top_p` limite la masse de probabilité considérée.

Paramètres fixes ne remplacent pas l'évaluation.

Sortie structurée

Un prompt demandant du JSON n'est pas un contrat.

1. Décoder
2. Vérifier la racine
3. Vérifier les clés
4. Vérifier types et domaines
5. Construire l'objet métier

Validation comme frontière

```
obj = json.loads(raw)
required = {"category", "summary", "next_action"}
if set(obj) != required:
    raise ValueError("unexpected schema")
if obj["category"] not in ALLOWED:
    raise ValueError("unknown category")
```

Reprise bornée, fallback ou escalade humaine.

Prompt = interface versionnée

- tâche et limites ;
- entrées délimitées ;
- sortie exacte ;
- catégories autorisées ;
- conduite si l'information manque ;
- exemples normaux et limites.

Versionner avec modèle, paramètres et corpus.

Charger un dataset

```
from datasets import load_dataset

ds = load_dataset("json", data_files={
    "train": "data/incidents_train.jsonl",
    "test": "data/incidents_test.jsonl",
})

clean = ds.filter(valid_row).map(normalize)
```

Le split et la révision font partie de l'expérience.

Fuite et contamination

Des tickets d'un même client ou issus du même modèle de texte peuvent se retrouver des deux côtés.

- grouper avant de séparer ;
- dédupliquer ;
- geler le test ;
- ne pas ajuster le prompt sur le test.

Embedding

Un texte devient un vecteur $x \in \mathbb{R}^d$.

$$\cos(q, x) = \frac{q^\top x}{\|q\| \|x\|}$$

Après normalisation : une multiplication matrice–vecteur donne tous les scores.

Top- k avec NumPy

```
docs = documents / np.clip(
    np.linalg.norm(documents, axis=1, keepdims=True),
    1e-12, None)
q = query / max(np.linalg.norm(query), 1e-12)
scores = docs @ q
idx = np.argpartition(scores, -k)[-k:]
idx = idx[np.argsort(scores[idx])[:, -1]]
```

Complexité de la recherche exacte

Pour n documents et dimension d :

- scores : $O(nd)$;
- sélection partielle : $O(n)$ en moyenne ;
- tri des candidats : $O(k \log k)$;
- scores : $O(n)$ d'espace.

L'index approximatif change le compromis.

RAG : six étapes

Ingestion → découpage → embeddings → index → récupération → génération.

Diagnostic

Mesurer si la preuve existe, est récupérée, puis est utilisée fidèlement.

Découper un corpus

- trop court : contexte perdu ;
- trop long : signal dilué ;
- recouvrement : couverture mais duplication ;
- métadonnées : source, date, droits, identifiant.

La bonne unité suit la structure documentaire.

Architecture remplaçable

Entrée → politique → prompt → TextModel → parseur → décision

Le modèle réel et le faux modèle implémentent le même contrat.

Tester sans réseau

```
class FakeModel:
    def generate(self, messages):
        return ('{"category": "paiement", '
                '"summary": "Double debit.", '
                '"next_action": "Verifier."}')

service = IncidentService(FakeModel())
assert service.analyze("double debit").category == "paiement"
```


Trois couches d'évaluation

1. **Composant** : JSON, langue, longueur, domaine.
2. **Tâche** : exactitude, fidélité, couverture, segments.
3. **Système** : latence, coût, erreurs, escalades, valeur métier.

Corpus d'évaluation

Une ligne = un cas indépendant et versionné.

- identifiant ;
- entrée ;
- cible ou grille ;
- segment ;
- sensibilité ;
- difficulté ;
- résultat détaillé par version.

Pas d'agrégat sans table détaillée.

Comparer deux versions

Même cas, deux versions : comparaison appariée.

- inspecter les régressions ;
- bootstrap au bon niveau ;
- masquer la version aux évaluateurs ;
- rapporter segments et incertitude.

Sécurité : frontières

- secret hors du code ;
- données minimisées avant transport ;
- documents traités comme données non fiables ;
- sortie validée ;
- permissions appliquées par le programme ;
- confirmation pour action sensible.

Prompt injection

« Ignore les instructions précédentes » peut être contenu dans un document.

Principe

Le texte n'obtient aucune autorité. Le code garde les permissions et la liste blanche des outils.

Résilience

- timeout explicite ;
- reprises seulement transitoires ;
- nombre maximal d'essais ;
- backoff et limitation de concurrence ;
- fallback et escalade ;
- journaux sans contenu sensible.

Performance

- lot : débit contre mémoire et latence ;
- padding : calcul gaspillé ;
- cache : clé complète et politique de données ;
- streaming : mesurer temps au premier token ;
- coût : tokens réels d'entrée et sortie.

Quand ne pas utiliser un LLM ?

- règle déterministe suffisante ;
- quatre classes stables avec données labellisées ;
- action irréversible non contrôlée ;
- frontière de données incompatible ;
- gain inférieur au coût système.

La baseline classique reste obligatoire.

Travail appliqué

1. grille de sélection ;
2. parseur adversarial ;
3. top- k cosinus ;
4. banc d'évaluation ;
5. revue sécurité et architecture.

Livrable : code, tests, table de mesures et décision argumentée.

Synthèse

- Séparer données, modèle, validation et décision.
- Versionner tous les artefacts.
- Évaluer par cas et par segment.
- Borner coûts et échecs.
- Le modèle propose ; le système autorise.

Source : Hugging Face Hub, Transformers et Datasets ; NumPy ; scikit-learn